

Privacy-Aware Task Scheduling in Satellite Edge Computing: A Game-Theoretic Framework

Ruizhi Wang[✉], Xiaolong Xu[✉], *Senior Member, IEEE*, Guangming Cui[✉], *Member, IEEE*,
Haipeng Dai[✉], *Senior Member, IEEE*, Lianyong Qi[✉], *Senior Member, IEEE*, Xuyun Zhang[✉], *Member, IEEE*,
and Wanchun Dou[✉], *Member, IEEE*

Abstract—The growing deployment of low Earth orbit (LEO) constellations equipped with onboard edge computing is making satellite edge computing (SEC) increasingly practical by enabling onboard processing for latency-sensitive tasks. In this setting, heterogeneous onboard computing capacities, bandwidth-limited connectivity, and resource contention make task scheduling indispensable for coordinating task placement and multi-hop forwarding. Conventional schedulers often optimize latency and energy first (tight deadlines, limited onboard energy, and measurable costs), whereas privacy is harder to quantify and thus under-modeled, potentially leading to higher exposure via fast multi-hop forwarding, untrusted execution choices, task co-location, etc. To address these conflicting objectives, this paper formulates a unified optimization framework and proposes two complementary scheduling schemes. We first introduce Task Scheduling Optimizer (TS-OPT), a centralized mixed-integer programming approach that provides an optimal reference for the proposed model. To overcome the scalability limitations of centralized solvers, we subsequently develop the Privacy-Aware Potential Game Scheduler (PAPGS), a decentralized algorithm that casts the problem as an exact potential game and converges in a finite number of best-response updates to a pure-strategy Nash equilibrium. Simulation results demonstrate

that PAPGS achieves a 0.18% optimality gap to TS-OPT in small-scale scenarios, and consistently attains lower unified system cost with a high completion rate at larger scales.

Index Terms—Satellite edge computing, privacy preservation, task scheduling, game theory.

I. INTRODUCTION

THE rapid proliferation of low Earth orbit (LEO) constellations is reshaping satellite–terrestrial integrated networks providing persistent global connectivity and moving computation closer to data sources [1]. Building on this trend, modern satellites are increasingly equipped with onboard multi-access edge computing (MEC) capabilities, extending their role beyond traditional relay communications and enabling latency-sensitive services such as emergency response, real-time monitoring, and time-critical analytics [2]. To support such services in practice, a task in satellite edge computing (SEC) systems is typically uplinked to a source satellite and then processed either on that satellite, forwarded through inter-satellite links (ISLs) to another satellite with available resources, or offloaded to a ground cloud center upon contact [3]. Since each task faces multiple execution and routing paths coupled with resource contention, the system must continuously optimize task placement and forwarding strategies. Under these constraints, providing reliable low-latency service depends critically on timely task scheduling that adapts to instantaneous constellation connectivity and resource availability. Nonetheless, developing such a scheduler is nontrivial due to the strong coupling between placement-and-routing decisions and mobility-induced link variations.

Specifically, post-uplink task scheduling in SEC is challenging because routing and execution placement are tightly coupled. Choosing a destination implicitly constrains the feasible multi-hop ISL path, while the path selection itself affects delay and energy through link contention and queuing. Under heavy traffic, this coupling intensifies because transmission and computation costs become strongly load-dependent, causing the end-to-end cost to escalate rapidly. In addition, decisions often face tight time budgets, making sub-second online decision making important for meeting stringent deadlines [4], [5]. As a result, solver-based centralized optimization is difficult to scale, and scalable online scheduling typically relies on simplified models or decentralized decisions, which may suffer degraded performance or stability under heavy traffic and heterogeneous

Received 24 January 2026; revised 13 April 2026; accepted 19 May 2026. Date of publication 22 May 2026; date of current version 1 September 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62372242, in part by the Natural Science Foundation of Jiangsu Province under Grant BK20240692, and in part by the Natural Science Foundation of the Jiangsu Higher Education Institutions of China under Grant 24KJB520021. (Corresponding author: Xiaolong Xu.)

Ruizhi Wang is with the School of Computer Science, School of Software, Nanjing University of Information Science and Technology, Nanjing 210044, China (e-mail: rzwdm3ky@163.com).

Xiaolong Xu is with the School of Computer Science, School of Software, Nanjing University of Information Science and Technology, Nanjing 210044, China, and also with the Jiangsu Province Engineering Research Center of Advanced Computing and Intelligent Services and the Jiangsu Collaborative Innovation Center of Atmospheric Environment and Equipment Technology, Nanjing 210044, China (e-mail: xlxu@nuist.edu.cn).

Guangming Cui is with the School of Computer Science, School of Software, Nanjing University of Information Science and Technology, Nanjing 210044, China, and also with the Laboratory of Symbolic Computation and Knowledge Engineering of Ministry of Education, Jilin University, Changchun 130012, China (e-mail: gcui@nuist.edu.cn).

Haipeng Dai and Wanchun Dou are with the State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing 210023, China (e-mail: haipengdai@nju.edu.cn; douwc@nju.edu.cn).

Lianyong Qi is with the College of Computer Science and Technology, China University of Petroleum (East China), Qingdao 266580, China (e-mail: lianyongqi@gmail.com).

Xuyun Zhang is with the School of Computing, Macquarie University, Sydney 2109, Australia (e-mail: xuyun.zhang@mq.edu.au).

Digital Object Identifier 10.1109/TDSC.2026.3696118

TABLE I
MAIN SYMBOLS USED IN THE SYSTEM AND PRIVACY-AWARE FORMULATION

Symbol	Definition
N	Total number of satellites
s_n	n -th satellite in the system
SC_n	Computing capacity of satellite s_n
TS_n	Set of tasks initially received by satellite s_n
$\mathcal{I}$	Global task set, $\mathcal{I} = \bigcup_{n=1}^N TS_n$
$s(i)$	Source satellite that initially receives task i
$\mu(i)$	User that generated task i
d_i	Data size of task i
Δd_i	Additional data size caused by encryption
t_i	Maximum tolerable delay of task i
c_i	Required CPU cycles per bit of task i
e_i	Encryption indicator of task i (1=encrypted, 0=plain)
k	Execution destination index; $k = 0$ denotes the ground cloud and $k \in \{1, \dots, N\}$ denotes satellite s_k
$\mathcal{A}$	Set of feasible directed inter-satellite links (ISLs)
$(u, v) \in \mathcal{A}$	Directed ISL from satellite s_u to satellite s_v
$H_{i,n,k}$	Ordered ISL path used by task $i \in TS_n$ when assigned to remote satellite s_k
$x_{i,n,k}$	$= 1$ if task $i \in TS_n$ is assigned to destination k
$y_{i,n,u,v}$	$= 1$ if task $i \in TS_n$ uses directed ISL (u, v)
$z_{i,j,k}$	Co-location indicator of tasks i, j on destination k
$\mathcal{R}_n$	Communication-state vector of satellite s_n
$\mathcal{R}$	Global communication-state collection of all satellites
$Q_{\max}$	Maximum queue capacity
$P_{n,\max}$	Maximum transmission power of satellite s_n
h_i	Hop count / path length of task i
λ_{server}	Weight of server-level co-location leakage
λ_{path}	Weight of path-induced leakage
λ_{enc}	Weight of non-encrypted-task leakage
$\lambda_{\text{loc}}(k)$	Trust weight of location k (larger = more trusted)
γ_{privacy}	Global privacy-importance coefficient
τ_{enc}	Per-bit encryption delay coefficient
η_{enc}	Per-bit encryption energy coefficient

and stringent latency requirements. Representative examples include PtrTasking for multi-connectivity-enabled MEC scheduling [15], hierarchical collaborative MEC optimization for service placement, task scheduling, and resource allocation [16], and combinatorial-auction-based dynamic distributed DNN inference scheduling in edge-cloud networks [17]. These studies show the importance of jointly considering computation allocation, network conditions, and resource contention, which becomes even more challenging in satellite edge computing.

B. Task Scheduling in Satellite Edge Computing

Task scheduling in SEC determines where tasks are executed and how scarce on-board resources are allocated under limited connectivity and service capacity. Recent studies have increasingly moved beyond stand-alone placement and coupled scheduling with higher-level orchestration factors. Tang et al. [9] jointly optimized service deployment and task scheduling, while Huang et al. [10] incorporated resource slicing into satellite-terrestrial edge scheduling. Fan et al. [18] further considered demand-driven scheduling for heterogeneous task requirements, and Yao et al. [19] developed a satellite-ground cooperative platform with layered scheduling for AI inference in large LEO constellations. Zhang et al. [11] additionally showed that data-dependent constraints and limited buffers can cause feasibility

issues such as deadlocks, motivating schedulers with explicit constraint handling.

A key difficulty of constellation-internal SEC scheduling is the tight coupling between execution placement and multi-hop forwarding. Li et al. [20] addressed routing stability in operational LEO networks. From the traffic-engineering perspective, Wu et al. [21] optimized path selection for low-latency satellite networking, and Zhang et al. [22] adapted segment-routing ideas to ground-station-assisted constellations for traffic engineering and fast reroute. Theeranantachai et al. [23] studied the effect of ground-satellite handovers on stateful forwarding, whereas Chen et al. [24] and Li et al. [25] highlighted scalability limits imposed by physical constraints and spatiotemporal supply-demand mismatch.

Overall, existing SEC schedulers still mainly optimize performance metrics such as delay, energy, and resource efficiency, while privacy exposure in post-uplink, constellation-internal scheduling remains under-explored.

C. Privacy-Aware Scheduling Methods

Given the decentralized and large-scale nature of SEC, scalable scheduling often relies on learning-based or game-theoretic methods. Mean-Field Games (MFG) are effective for massive-population decision making: Wu et al. [26] applied MFG to load balancing in industrial MEC, and Wang et al. [27] surveyed MFG-based control in space-air-ground networks. Ji et al. [28] studied cooperative multi-agent DRL for satellite edge computing, while Hsieh et al. [29] showed that potential-game formulations can provide decentralized updates with convergence guarantees. These studies indicate the importance of scalability and stability for SEC scheduling.

Privacy and security have also been studied in edge and satellite-integrated systems, but mostly outside constellation-internal scheduling [30]. Existing privacy-aware work mainly focuses on uplink/offloading privacy, especially location privacy [12], [13]. System-level protection has been investigated through authentication [31], auditing [32], trust evaluation [33], blockchain/zero-trust-based secure data sharing [34], and privacy-preserving federated learning [35].

However, these studies still do not jointly model co-location risk, hop-induced exposure, encryption-dependent risk, and destination trust in the same post-uplink constellation-internal scheduling space, nor do they provide a decentralized scheduler with provable convergence for this setting. Our work fills this gap through a privacy-aware exact-potential-game scheduler for SEC.

III. SYSTEM DESCRIPTION AND PROBLEM FORMULATION

Throughout the remainder of the paper, i and j denote task indices, n denotes the source satellite that initially receives task i , and $k \in \{0, 1, \dots, N\}$ denotes the selected execution destination, where $k = 0$ represents the ground cloud and $k \geq 1$ represents satellite s_k . We use (u, v) exclusively for directed ISL endpoints, write $s(i)$ for the source satellite of task i , and use $\mu(i)$ for the user that generated task i . To avoid symbol overloading,

$\mathcal{R}_n$ and $\mathcal{R}$ denote the per-satellite and global communication states, whereas $R_{u,v}$ and $R_{i,j}^k$ denote achievable link rates.

Motivated by the above example mentioned in Section I, we now translate the qualitative trade-off between service performance and privacy exposure into a tractable optimization problem for satellite edge computing. Specifically, we first describe the satellite–terrestrial integrated architecture, then detail the transmission and computation models, and finally extend the original MINLP by introducing a privacy-leakage model that is structurally parallel to the delay and energy models.

A. System Overview

The system consists of N satellites, denoted by $S = \{s_1, s_2, \dots, s_N\}$. Each satellite s_n is equipped with on-board MEC capability SC_n and maintains a set of tasks TS_n that are directly offloaded from ground users or base stations (BSs) (i.e., excluding relayed tasks) [1], [9], [10]. We define the global task set as $\mathcal{I} = \bigcup_{n=1}^N TS_n$. Each task $i \in TS_n$ is generated by a ground user indexed by $\mu(i)$ and is characterized by (d_i, t_i, c_i, e_i) , i.e., input size d_i , deadline t_i , required CPU cycles per bit c_i , and encryption-status indicator e_i .

The spatial position of satellite s_n is obtained from the spherical-to-Cartesian transformation as follows [4], [20]:

$$\begin{aligned} x_{s_n} &= (R_{\text{Earth}} + h_n) \cos(\lambda_n) \cos(\varphi_n), \\ y_{s_n} &= (R_{\text{Earth}} + h_n) \sin(\lambda_n) \cos(\varphi_n), \\ z_{s_n} &= (R_{\text{Earth}} + h_n) \sin(\varphi_n), \end{aligned} \quad (1)$$

where R_{Earth} is the Earth's radius, h_n is the altitude of satellite s_n , and λ_n and φ_n represent the longitude and latitude of satellite s_n , respectively. The longitude λ_n of satellite s_n is determined by the orbital phase offset F_n , which is introduced to evenly space satellites on the same orbit. Specifically, the longitude λ_n is given by [4]:

$$\lambda_n = \lambda_0 + F_n \cdot \Delta\lambda, \quad (2)$$

where λ_0 is the reference longitude for the orbit, F_n is the orbital phase offset for satellite s_n , and $\Delta\lambda$ is the angular spacing between consecutive satellites in the same orbit. This phase offset F_n ensures that the satellites are uniformly distributed on the orbit, improving coverage uniformity and reducing interference between the satellites.

Each satellite keeps two queues: a transmission queue $Q_{n,\text{trans}}$ for tasks waiting to be forwarded, and a computation queue $Q_{n,\text{comp}}$ for tasks scheduled for local execution. The ground segment consists of a set of users and a set of ground cloud centers. Tasks can be (i) computed locally at the source satellite, (ii) forwarded to other satellites via ISLs, or (iii) forwarded to a ground cloud center when a feasible ground–satellite link exists. In this work, we focus on the task scheduling after tasks are uploaded to the satellites, specifically handling the task assignment and processing within the satellite network. We consider an integrated SEC architecture, including LEO satellites, BSs, mobile devices (MDs), ground cloud centers, and satellite-mounted MEC servers.

The ground cloud centers are placed at known geographic coordinates (ϕ_g, λ_g) , where ϕ_g is the latitude and λ_g is the longitude, and these are mapped to Cartesian coordinates (x_g, y_g, z_g) by [4]:

$$\begin{aligned} x_g &= R_{\text{Earth}} \cos(\phi_g) \cos(\lambda_g), \\ y_g &= R_{\text{Earth}} \cos(\phi_g) \sin(\lambda_g), \\ z_g &= R_{\text{Earth}} \sin(\phi_g), \end{aligned} \quad (3)$$

so that the satellite–ground distance can be computed in 3D space.

For task $i \in TS_n$, the final scheduling decision is represented by the binary assignment variables $x_{i,n,k}$, where $k \in \{0, 1, \dots, N\}$ denotes the selected execution destination: $k = 0$ corresponds to the ground cloud, $k = n$ corresponds to local execution at the source satellite, and $k \in \{1, \dots, N\} \setminus \{n\}$ corresponds to execution at a remote satellite reached via ISLs. For a remotely executed task assigned to satellite s_k , the corresponding ISL path is denoted by

$$H_{i,n,k} = \{(n, p_{i,n,k}^1), (p_{i,n,k}^1, p_{i,n,k}^2), \dots, (p_{i,n,k}^{f-1}, k)\},$$

where $p_{i,n,k}^1, p_{i,n,k}^2, \dots, p_{i,n,k}^{f-1}$ are relay satellites and the elements of $H_{i,n,k}$ are directed ISL segments.

B. Link and Rate Model

The laser ISLs with free-space path loss can be denoted as [1]:

$$FPL_{o,d} = \left(\frac{4\pi d_{o,d} f}{c} \right)^2, \quad (4)$$

where $d_{o,d}$ is the 3D distance between satellites s_o and s_d , f is the carrier frequency, and c is the speed of light. The distance is

$$d_{o,d} = \sqrt{(x_{s_d} - x_{s_o})^2 + (y_{s_d} - y_{s_o})^2 + (z_{s_d} - z_{s_o})^2}. \quad (5)$$

Let p_o be the transmit power of satellite s_o , G_o and G_d the antenna gains, T_s the system noise temperature, N' an additional noise term, and $(\frac{E_b}{N_0^{\text{eff}}})_{\text{req}}$ the signal-to-noise ratio (SNR) threshold for the chosen modulation and coding scheme (MCS). Define $N_0^{\text{eff}} = k_B T_s + N'$, where k_B is the Boltzmann constant.

$$R_{o,d} = \frac{p_o G_o G_d}{FPL_{o,d} N_0^{\text{eff}} \left(\frac{E_b}{N_0^{\text{eff}}} \right)_{\text{req}} M}. \quad (6)$$

For multi-hop forwarding, per-hop rates are obtained from (6) and used to compute the hop-by-hop transmission delay and energy.

The satellite–ground path loss $PL_{i,j}$ is calculated as follows [1], [36]:

$$\begin{aligned} PL_{i,j} &= 20 \log_{10}(d) + 20 \log_{10}(f) + 20 \log_{10} \left(\frac{4\pi}{c} \right) \\ &\quad + L_{\text{atm}} + L_{\text{rain}}, \end{aligned} \quad (7)$$

where d is the 3D distance between satellite i and ground node j , f is the carrier frequency of the communication link. The additional terms L_{atm} and L_{rain} represent the atmospheric attenuation and rain attenuation, respectively. Atmospheric loss L_{atm} is modeled as $L_{\text{atm}} = \alpha_{\text{atm}} d$, where α_{atm} is the atmospheric

absorption coefficient that depends on factors like humidity and pressure, and d is the distance between the satellite and the ground node. Rain loss L_{rain} is modeled as $L_{\text{rain}} = \gamma_{\text{rain}}d$, where γ_{rain} is the rain attenuation coefficient that depends on the intensity of the rainfall and the carrier frequency.

The channel gain $h_{i,j}^k$ for channel k is then given by:

$$h_{i,j}^k = G_t G_r \cdot 10^{-PL_{i,j}/10}, \quad (8)$$

where G_t and G_r represent the antenna gains of the transmitter (satellite) and receiver (ground node).

In this system, Power-domain Non-Orthogonal Multiple Access (NOMA) is adopted, where multiple satellites share the same communication channel k . The satellites are ordered by their effective channel gains, and the signals are decoded using Successive Interference Cancellation (SIC) at the ground receiver [37]. The achievable rate for the i -th decoded satellite is given by:

$$R_{i,j}^k = W \log_2 \left(1 + \frac{P_i h_{i,j}^k}{N_0 + \sum_{l=i+1}^{N_{\text{sats}}} P_l h_{l,j}^k} \right), \quad (9)$$

where W is the bandwidth of the channel, P_i is the transmit power of satellite i , $h_{i,j}^k$ is the channel gain between satellite s_i and ground node j on channel k , N_0 is the receiver noise power, and P_l is the transmit power of satellite l (for $l > i$, representing the interference from satellites decoded later). The summation $\sum_{l=i+1}^{N_{\text{sats}}} P_l h_{l,j}^k$ represents the total interference from satellites that are decoded after satellite i .

The interference is modeled based on the effective channel gain ordering. Specifically, satellites with higher channel gains are decoded first, and weaker signals are treated as interference in later decoding stages. Therefore, the interference experienced by satellite i is the sum of the signals from satellites $l > i$, as they are decoded in later stages of SIC.

We define the communication status of satellite s_n as

$$\mathcal{R}_n = \{R_{n,0}, R_{n,1}, \dots, R_{n,m}, \dots, R_{n,N}\},$$

where $R_{n,0}$ denotes the achievable transmission rate from satellite s_n to the ground cloud, and $R_{n,m}$ represents the achievable rate between satellite s_n and satellite s_m through ISLs. Each rate term is determined by the corresponding link distance, channel gain, and interference level under the current network conditions.

For convenience, we collect the communication states of all satellites as

$$\mathcal{R} = \{\mathcal{R}_1, \mathcal{R}_2, \dots, \mathcal{R}_N\},$$

which jointly characterizes the instantaneous transmission capability of the entire satellite network, including both satellite-ground and inter-satellite communications.

C. Delay and Energy Computation Model

1) *Delay*: For task $i \in TS_n$, the transmission delay is given by

$$D_{\text{trans},i} = \begin{cases} 0, & \text{if } x_{i,n,n} = 1, \\ \frac{d_i + \Delta d_i}{R_{n,0}} + \frac{l_{n,0}}{c} + D_{\text{que}}, & \text{if } x_{i,n,0} = 1, \\ \sum_{(u,v) \in H_{i,n,k}} \left(\frac{d_i + \Delta d_i}{R_{u,v}} + \frac{l_{u,v}}{c} + D_{\text{que}} \right), & \text{if } x_{i,n,k} = 1 \text{ with } k \in \{1, \dots, N\} \setminus \{n\}, \end{cases} \quad (10)$$

where the first, second, and third cases correspond to local execution at the source satellite ($x_{i,n,n} = 1$), satellite-ground offloading ($x_{i,n,0} = 1$), and inter-satellite multi-hop forwarding ($x_{i,n,k} = 1$ with $k \in \{1, \dots, N\} \setminus \{n\}$), respectively. Here, D_{que} denotes the average queuing delay, $R_{u,v}$ is the achievable rate on directed ISL (u, v) , and $l_{u,v}$ is the propagation distance of that segment [9], [20]. The term Δd_i accounts for the additional data size introduced by encryption. When the task is encrypted, Δd_i increases the total number of transmitted bits to $d_i + \Delta d_i$, thereby increasing the transmission delay whenever communication is involved.

After task i reaches its selected execution destination $k \in \{0, 1, \dots, N\}$, it is executed with the per-task available CPU

$$f_{\text{ava},i} = \frac{f_k}{N_k}. \quad (11)$$

where f_k denotes the computation capacity of the selected destination k (with $f_0 = GC$ for the ground cloud and $f_k = SC_k$ for satellite s_k , $k \geq 1$), and N_k is the number of tasks queued at destination k .

The execution delay is

$$D_{\text{exec},i} = \frac{(d_i + \Delta d_i)c_i}{f_{\text{ava},i}}. \quad (12)$$

2) *Energy*: The transmission energy is given by

$$E_{\text{trans},i} = \begin{cases} 0, & \text{if } x_{i,n,n} = 1, \\ \frac{P_{n \rightarrow 0}^{\text{tx}}(d_i + \Delta d_i)}{R_{n,0}}, & \text{if } x_{i,n,0} = 1, \\ \sum_{(u,v) \in H_{i,n,k}} \frac{P_{u,v}^{\text{tx}}(d_i + \Delta d_i)}{R_{u,v}}, & \text{if } x_{i,n,k} = 1 \text{ with } k \in \{1, \dots, N\} \setminus \{n\}, \end{cases} \quad (13)$$

where the first, second, and third cases correspond to local execution at the source satellite ($x_{i,n,n} = 1$), satellite-ground offloading ($x_{i,n,0} = 1$), and inter-satellite multi-hop forwarding ($x_{i,n,k} = 1$ with $k \in \{1, \dots, N\} \setminus \{n\}$), respectively. Here, $P_{n \rightarrow 0}^{\text{tx}}$ denotes the transmission power used by source satellite s_n when offloading to the ground cloud, and $P_{u,v}^{\text{tx}}$ denotes the transmission power used over ISL (u, v) .

The computation energy is

$$E_{\text{exec},i} = \varepsilon (f_{\text{ava},i})^3 D_{\text{exec},i}, \quad (14)$$

with ε the energy-efficiency coefficient.

D. Threat Model and Privacy-Aware Leakage Model

We consider a post-uplink privacy threat model for satellite edge computing in which the privacy principals are the ground users that generate tasks. In the considered system, ground users

submit privacy-sensitive tasks to source satellites, which are the first infrastructure nodes that receive uploaded tasks. Tasks may then be forwarded by relay satellites over ISLs and finally processed by execution nodes, including remote satellites and the ground cloud. Task placement and forwarding decisions are determined by the scheduling mechanism. The privacy objective is to reduce the exposure of both task payloads and task-associated metadata, including routing choices, destination choices, timing patterns, and associations among tasks generated by the same user.

The scheduling mechanism, whether realized by the centralized TS-OPT solver or by the distributed decision process induced by PAPGS, is treated as protocol-compliant and non-adversarial. By contrast, source satellites, relay satellites, and candidate execution nodes are modeled as honest-but-curious and non-colluding infrastructure nodes. Such nodes follow the prescribed forwarding and execution procedure, but may attempt to infer private information from the tasks they directly receive, relay, or execute. For any infrastructure node v , the observable information regarding task i is restricted to the content and metadata locally exposed along that interaction, rather than the full global communication state $\mathcal{R}$. Formally, letting $\mathcal{O}_v(i)$ denote the information observable to node v for task i , we require $\mathcal{O}_v(i) \subseteq \{\text{payload of } i, \text{ metadata of } i\}$, and $\mathcal{O}_v(i) = \emptyset$ if node v does not directly handle i .

Under this threat model, privacy risks arise through both content exposure and scheduling-induced side information. For an unencrypted task, the source, relay, or execution node that directly handles the task may observe its payload together with its metadata. For an encrypted task, payload confidentiality is protected, but side information such as the selected route, the number of traversed hops, repeated destination choices, timing patterns, and the co-location of multiple tasks from the same user at the same execution node may still be exposed. Trust heterogeneity across candidate execution locations is captured by the destination-dependent weight $\lambda_{\text{loc}}(k) \in [0, 1]$, where a larger value indicates a more trusted execution location. In particular, $\lambda_{\text{loc}}(0) > \lambda_{\text{loc}}(k)$ for $k \in \{1, \dots, N\}$, which models the ground cloud ($k = 0$) as more strongly protected than in-orbit execution nodes.

Accordingly, privacy leakage is not modeled as a strict information-theoretic quantity, but as a scheduling-oriented composite privacy-risk score that compares the relative exposure induced by alternative joint placement-and-routing decisions. The modeled exposure channels are aligned with the SEC scheduling process. Server-level co-location leakage captures association risk when multiple tasks from the same user are assigned to the same execution destination. Path-induced leakage captures relay exposure introduced by multi-hop ISL forwarding. Encryption-dependent leakage captures the higher residual risk of plaintext transmission. Execution-location leakage captures destination-side exposure caused by heterogeneous trust levels.

Based on the above threat setting, we place privacy on the same footing as delay and energy by formulating a task-level privacy leakage score that aggregates four commonly observed sources of leakage in SEC scenarios [6]:

- 1) **Server-level co-location leakage** (same-user tasks processed on the same node);
- 2) **Path-induced leakage** (long, unstable, or multi-hop routes expose data to more nodes);
- 3) **Encryption-dependent leakage** (unencrypted tasks are riskier);
- 4) **Execution-location leakage** (processing on low-trust nodes is riskier).

In this paper, Leak_i is modeled as a composite privacy-risk score for scheduling, rather than as a strict information-theoretic leakage measure. Its additive form captures four complementary exposure channels induced by the same placement-and-routing decision: server-level co-location reflects association risk among tasks from the same user, path-induced leakage reflects relay exposure introduced by multi-hop forwarding, encryption-dependent leakage reflects the higher vulnerability of plaintext transmission, and execution-location leakage reflects heterogeneous trust across candidate execution destinations. Accordingly, the leakage score is used to compare the relative privacy exposure of alternative scheduling decisions within a unified optimization framework.

Server-level co-location leakage occurs when multiple tasks from the same user are assigned to the same destination node, which increases the risk of profiling via cross-task inference. Consider two tasks i and j generated by the same user, i.e., $\mu(i) = \mu(j)$, and suppose that both are assigned to the same destination $k \in \{0, 1, \dots, N\}$. Here, k denotes the selected execution destination, where $k = 0$ represents the ground cloud and $k \geq 1$ represents satellite s_k . We use this destination-index notation consistently with the binary assignment variables $x_{i,s(i),k}$ and $x_{j,s(j),k}$ introduced below, so that the privacy-leakage model is expressed in the same assignment space as the delay and energy models.

The risk that the destination node can infer additional profile information from multiple tasks of the same user increases when tasks from the same user are assigned to the same destination. To capture this, we introduce auxiliary variables $z_{i,j,k} \in \{0, 1\}$ for all $i, j \in \mathcal{I}$, $i < j$, and $k \in \{0, 1, \dots, N\}$:

$$z_{i,j,k} \leq x_{i,s(i),k}, \quad (15a)$$

$$z_{i,j,k} \leq x_{j,s(j),k}, \quad (15b)$$

$$z_{i,j,k} \geq x_{i,s(i),k} + x_{j,s(j),k} - 1, \quad (15c)$$

where $s(i)$ denotes the source satellite that initially receives task i (i.e., $i \in TS_{s(i)}$). These three constraints define the auxiliary variable $z_{i,j,k}$, which reflects whether tasks i and j are assigned to the same destination node k (such as the same satellite or ground cloud center). The first constraint (15a) ensures that task i must be assigned to destination k for there to be a potential co-location leakage risk. If task i is not assigned to destination k , then $z_{i,j,k} = 0$. The second constraint (15b) is similar, ensuring that task j must also be assigned to destination k for co-location leakage risk to exist. (15c) ensures that only when both tasks i and j are assigned to the same destination node k , $z_{i,j,k}$ will be 1, otherwise, it will be 0. Through these three constraints, the model accurately captures the effect of task assignment on privacy leakage risk.

Then the co-location leakage of task i is

$$L_i^{\text{srv}} = \lambda_{\text{server}} \sum_{\substack{j \in \mathcal{I} \\ j \neq i \\ \mu(j) = \mu(i)}} \sum_{k=0}^N z_{i,j,k}. \quad (16)$$

The co-location leakage of task i , denoted as L_i^{srv} , quantifies the privacy leakage risk when multiple tasks from the same user are assigned to the same destination node. In this formula, λ_{server} is a weight factor that represents the risk of privacy leakage associated with tasks being assigned to the same destination node. The set $\mathcal{I}$ contains all the tasks in the system, and $\mu(i)$ denotes the user to whom task i belongs. L_i^{srv} is calculated by summing over all tasks j that belong to the same user as task i (i.e., $\mu(j) = \mu(i)$) and checking whether both tasks are assigned to the same destination node k . The term λ_{server} scales the leakage risk, reflecting the importance of task co-location in the privacy model.

Path-induced leakage captures the privacy exposure introduced by multi-hop forwarding, since each additional hop potentially reveals the task to another relay node [5]. We quantify this exposure by the hop count of the selected route for task i :

$$h_i = \sum_{(u,v) \in \mathcal{A}} y_{i,s(i),u,v}, \quad (17)$$

where $\mathcal{A}$ is the set of feasible directed links. Here, h_i counts only the number of traversed ISLs, since $\mathcal{A}$ denotes the set of feasible directed ISLs. A longer route implies a greater number of intermediate nodes that could potentially observe the task. The path-induced leakage of task i is then modeled as:

$$L_i^{\text{path}} = \lambda_{\text{path}} h_i. \quad (18)$$

The path-induced leakage increases with the hop count h_i . The parameter λ_{path} is a weight factor that governs how sensitive the system is to privacy risks caused by longer paths. Therefore, tasks that traverse more hops are at a higher risk of being observed by unauthorized nodes, and thus, this leakage model directly relates the task's route length to its privacy exposure.

Encryption-dependent leakage accounts for the residual risk when a task is transmitted without encryption, as plaintext data are more vulnerable to interception during forwarding [14]. We use a binary encryption-status indicator e_i to capture whether task i is protected by encryption. The corresponding leakage penalty is modeled as:

$$L_i^{\text{enc}} = \lambda_{\text{enc}}(1 - e_i). \quad (19)$$

Encryption is an effective method to reduce privacy risks. A task without encryption faces a privacy risk, which is quantified by L_i^{enc} , a penalty factor. This leakage term penalizes non-encrypted tasks, which is important because they are more vulnerable to interception or unauthorized access during transmission.

To account for the extra cost of encryption, we also add the encryption overhead to the task's delay and energy consumption:

$$D_{\text{enc},i} = e_i \tau_{\text{enc}}(d_i + \Delta d_i), \quad (20a)$$

$$E_{\text{enc},i} = e_i \eta_{\text{enc}}(d_i + \Delta d_i). \quad (20b)$$

The encryption indicator e_i is treated as an exogenous task attribute specified before scheduling, rather than as a decision variable of the scheduler. Encryption is incorporated through its system-level scheduling impact, with encrypted tasks incurring additional delay and energy overhead and non-encrypted tasks incurring a larger privacy-risk penalty.

These equations capture the delay and energy overhead introduced by encryption, parameterized by the per-bit coefficients τ_{enc} and η_{enc} . Thus, encryption not only improves privacy but also introduces additional computational costs, which must be considered in the overall optimization.

Execution-location leakage reflects destination trust heterogeneity, where executing a task on a less trusted node incurs a higher privacy risk. We parameterize this with a per-location trust weight $\lambda_{\text{loc}}(k) \in [0, 1]$, where a larger value means destination k is more trusted. To keep this component non-negative and directly additive with the others, we define

$$L_i^{\text{loc}} = \sum_{k=0}^N (1 - \lambda_{\text{loc}}(k)) x_{i,s(i),k}, \quad (21)$$

so executing task i at a highly trusted location (large $\lambda_{\text{loc}}(k)$) yields a smaller contribution to L_i^{loc} , while less trusted locations (small $\lambda_{\text{loc}}(k)$) incur higher leakage. We normalize $\lambda_{\text{loc}}(k)$ across candidate execution locations so that L_i^{loc} remains non-negative and directly comparable across locations; concrete trust-weight settings are specified in the evaluation configuration [6], [14].

The overall privacy leakage score of task i is the weighted sum of the four components:

$$\text{Leak}_i = L_i^{\text{srv}} + L_i^{\text{path}} + L_i^{\text{enc}} + L_i^{\text{loc}}. \quad (22)$$

The total privacy leakage score combines the co-location leakage (L_i^{srv}), path-induced leakage (L_i^{path}), encryption-dependent leakage (L_i^{enc}), and execution-location leakage (L_i^{loc}). The weighted sum of these components determines the overall risk to privacy for each task. The goal is to minimize this total leakage, balancing privacy with other system objectives like delay and energy consumption.

E. Decision Variables and Constraint Definitions

We keep the original assignment and routing variables. Specifically, $x_{i,n,k} \in \{0, 1\}$ indicates whether task $i \in TS_n$ is assigned to destination $k \in \{0, 1, \dots, N\}$; $y_{i,n,u,v} \in \{0, 1\}$ indicates whether task $i \in TS_n$ uses link $(u, v) \in \mathcal{A}$; and $z_{i,j,k} \in \{0, 1\}$ is the co-location indicator defined above.

The one-hot assignment remains

$$\sum_{k=0}^N x_{i,n,k} = 1, \quad \forall i \in TS_n, \forall n. \quad (23)$$

Flow conservation also remains

$$\begin{aligned} & \sum_{(v,w) \in \mathcal{A}} y_{i,n,v,w} - \sum_{(u,v) \in \mathcal{A}} y_{i,n,u,v} \\ &= \begin{cases} 1 - x_{i,n,0} - x_{i,n,n}, & v = n, \\ -x_{i,n,v}, & v \in \{1, \dots, N\} \setminus \{n\}, \end{cases} \end{aligned} \quad (24)$$

for all tasks $i \in TS_n$ and all satellite nodes $v \in \{1, \dots, N\}$. In particular, the source node injects one unit of flow only when task i is forwarded to a remote satellite, injects no ISL flow when the task is offloaded directly to the ground cloud, and also injects no ISL flow under local execution. Any satellite node $v \neq n$ absorbs one unit of flow only when it is selected as the remote execution destination.

The co-location constraints are given in (15). Queue, power, and compute-capacity constraints are unchanged:

$$D_{\text{trans},i} + D_{\text{exec},i} + D_{\text{enc},i} \leq t_i, \quad (25a)$$

$$P_{n,i} \leq P_{n,\max}, \quad (25b)$$

$$\sum_{i \in Q_{n,\text{comp}}} f_{\text{ava},i} \leq SC_n, \quad (25c)$$

$$|Q_{n,\text{trans}}| \leq Q_{\max}, \quad |Q_{n,\text{comp}}| \leq Q_{\max}. \quad (25d)$$

Constraints (25) enforce four operational requirements. Constraint (25a) enforces per-task deadline feasibility by bounding the end-to-end latency (transmission, execution, and encryption overhead). Constraint (25b) enforces per-node peak transmit power, where $P_{n,i}$ denotes the transmit power used at satellite s_n when forwarding task i on its outgoing link (and equals zero if i is not forwarded by s_n). Constraint (25c) limits the aggregate CPU shares assigned to tasks in the computation queue $Q_{n,\text{comp}}$ by the on-board MEC capacity SC_n . Finally, (25d) bounds the cardinalities of both the transmission and computation queues to reflect finite buffers and prevent drops/instability.

F. Privacy-Aware Problem Formulation

Let the original per-satellite accumulated delay and energy be

$$\text{Delay}_n = \sum_{i \in TS_n} (D_{\text{trans},i} + D_{\text{exec},i} + D_{\text{enc},i}), \quad (26)$$

$$\text{Energy}_n = \sum_{i \in TS_n} (E_{\text{trans},i} + E_{\text{exec},i} + E_{\text{enc},i}). \quad (27)$$

We now add the task-level privacy leakage (22). The privacy-aware optimization problem is

$$\text{Objective} : \min \sum_{n=1}^N (\alpha \cdot \text{Delay}_n + \beta \cdot \text{Energy}_n) \quad (28a)$$

$$+ \gamma_{\text{privacy}} \sum_{i \in \mathcal{I}} \text{Leak}_i$$

s.t. Assignment constraint (23),

Flow conservation (24),

Co-location linearization (15),

Resource constraints (25),

$$x_{i,n,k}, y_{i,n,u,v}, z_{i,j,k} \in \{0, 1\}. \quad (28b)$$

In (28a), α and β are nonnegative trade-off weights that balance latency and energy, while γ_{privacy} controls the importance of privacy leakage. The formulation reuses the assignment and routing variables $x_{i,n,k}$ and $y_{i,n,u,v}$ so that delay,

energy, and privacy are optimized over a common decision space. The co-location term is linearized at the model level in (15), whereas the solver-based implementation of TS-OPT is described in Section IV-A. Together with the one-hot assignment (23), flow conservation (24), and resource constraints (25), these conditions characterize the feasible region of the optimization model.

IV. ALGORITHM

A. TS-OPT: Centralized Solver-Based Scheduling

We begin with a centralized scheduling method, termed TS-OPT. TS-OPT treats privacy-aware scheduling as a joint optimization problem over both execution placement and forwarding decisions. Given the current constellation connectivity and resource states, TS-OPT searches for a globally best schedule by minimizing the unified system cost in (28a), which explicitly balances latency, energy consumption, and task-level privacy leakage under the same decision space.

At a high level, TS-OPT proceeds by selecting, for each task, an execution destination (local satellite, a remote satellite reachable through ISLs, or a ground cloud when contact is available) and simultaneously choosing a feasible multi-hop route when remote execution is selected. The optimization enforces standard feasibility requirements—each task is assigned to exactly one destination, any inter-satellite forwarding forms a valid end-to-end path, and the resulting schedule respects the operational limits on delay, transmission power, computing capacity, and queuing. Privacy is embedded directly into the objective through the leakage model in (22), so that decisions that increase exposure (e.g., longer paths, less trusted destinations, or risky co-location patterns) are penalized together with delay and energy.

TS-OPT is implemented using a mixed-integer optimization solver (Gurobi) under a branch-and-bound framework. The resulting schedules provide a centralized, model-grounded solution that explicitly captures the coupled placement–routing trade-off under privacy risk. In the next subsection, we move to a decentralized design and develop PAPGS to achieve comparable scheduling behavior with substantially lower online decision overhead.

TS-OPT is implemented in Gurobi as a solver-based mixed-integer quadratic program reformulation of the privacy-aware scheduling problem. The quadratic structure is induced by the co-location leakage component, whereas the adopted constraints remain linear. In the following evaluation, a TS-OPT solution is reported as optimal only when the solver terminates with status GRB.OPTIMAL and the final optimality gap does not exceed 10^{-6} ; otherwise, the returned solution is treated as the best incumbent solution rather than a certified optimum.

B. Privacy-Aware Potential-Game Scheduler (PAPGS)

In this section, we design a decentralized scheduler by viewing each task as a player in a congestion game. We first rewrite the delay and energy in a congestion-based form, then define

the game, prove that it is an exact potential game, and finally present a best-response algorithm.

TS-OPT directly solves the privacy-aware scheduling formulation in Section III-F. PAPGS, by contrast, uses a congestion-based abstraction of the load-dependent delay and energy terms, so that the resulting decentralized interaction admits exact-potential analysis while retaining the main routing and execution congestion effects of the original model.

1) *Congestion-Based Representation of Delay and Energy:* To align the centralized formulation in Section III-F with a congestion-game structure, we represent the load-dependent delay and energy as sums of facility congestion costs plus load-independent constants. This is a modelling step: the functions introduced below can be calibrated from the physical expressions in (10), (12), (13), and (14).

Let $n_{u,v}(s)$ denote the number of tasks that use link $(u, v) \in \mathcal{A}$ under a joint strategy profile s , and let $n_k(s)$ denote the number of tasks finally assigned to destination $k \in \{0, 1, \dots, N\}$. For a task i with strategy $s_i = (k_i, \mathcal{P}_i)$, we write

$$D_{\text{trans},i}(s) = D_i^{\text{prop}} + \sum_{(u,v) \in \mathcal{P}_i} T_{u,v}(n_{u,v}(s)), \quad (29a)$$

$$E_{\text{trans},i}(s) = E_i^{\text{prop}} + \sum_{(u,v) \in \mathcal{P}_i} P_{u,v}(n_{u,v}(s)), \quad (29b)$$

$$D_{\text{exec},i}(s) = G_{k_i}(n_{k_i}(s)), \quad (29c)$$

$$E_{\text{exec},i}(s) = H_{k_i}(n_{k_i}(s)). \quad (29d)$$

More specifically, the congestion functions in (29) are motivated by the original physical expressions in Section III. The link-delay term $T_{u,v}(n)$ abstracts the load-dependent part of per-hop transmission delay, including both effective-rate degradation and queue growth on link (u, v) , as motivated by (10). Likewise, $P_{u,v}(n)$ abstracts the corresponding load-dependent transmission-energy term in (13). At the execution side, (12) and (14) imply that when more tasks are assigned to destination k , the available CPU share $f_{\text{ava},i} = f_k/N_k$ decreases, which increases execution delay and execution energy; this motivates the nondecreasing node functions $G_k(n)$ and $H_k(n)$. In contrast, propagation delay, encryption overhead, and other task-specific constants are separated into the load-independent terms D_i^{prop} , E_i^{prop} , $D_{\text{enc},i}$, and $E_{\text{enc},i}$. Therefore, the potential-game layer preserves the monotone load effects of the physical model while abstracting detailed waveform-level and channel-state-level dependencies into fitted congestion functions. The exact-potential analysis is established for this calibrated monotone congestion surrogate, rather than being claimed as a literal symbolic rewriting of every lower-layer physical expression.

2) *Game Formulation:* We model the privacy-aware scheduling problem as a finite strategic game

$$\mathcal{G} = (\mathcal{U}, \{\mathcal{A}_i\}_{i \in \mathcal{U}}, \{Z_i\}_{i \in \mathcal{U}}), \quad (30)$$

where $\mathcal{U}$ is the set of players, $\mathcal{A}_i$ is the strategy set of player i , and Z_i is its cost function.

We treat each task as a player, i.e., $\mathcal{U} = \mathcal{I}$. Accordingly, the strategy space of player i is $\mathcal{A}_i$, and a generic strategy is

$$s_i = (k_i, \mathcal{P}_i),$$

where $k_i \in \{0, 1, \dots, N\}$ is the destination ($k_i = 0$ for the ground cloud and $k_i \geq 1$ for satellite s_{k_i}), and $\mathcal{P}_i$ is a feasible path from the source satellite $s(i)$ to k_i . A joint strategy profile s is said to be feasible if the induced routing and assignment variables satisfy (23), (24), and the resource constraints (25).

Using the notation in Section III-D, the four leakage components under a profile s can be written as

$$L_i^{\text{srv}}(s) = \lambda_{\text{server}} \sum_{\substack{j \neq i \\ \mu(j) = \mu(i)}} \mathbf{1}[k_j = k_i], \quad (31a)$$

$$L_i^{\text{path}}(s) = \lambda_{\text{path}} h_i, \quad h_i = \sum_{(u,v) \in \mathcal{A}} y_{i,s(i),u,v}, \quad (31b)$$

$$L_i^{\text{enc}}(s) = \lambda_{\text{enc}} (1 - e_i), \quad (31c)$$

$$L_i^{\text{loc}}(s) = 1 - \lambda_{\text{loc}}(k_i). \quad (31d)$$

Recalling the centralized objective in (28a), the cost of player i under joint strategy s is

$$Z_i(s) = \alpha \text{Delay}_i(s) + \beta \text{Energy}_i(s) + \gamma_{\text{privacy}} \text{Leak}_i(s), \quad (32)$$

where

$$\text{Delay}_i(s) = D_{\text{trans},i}(s) + D_{\text{exec},i}(s) + D_{\text{enc},i}, \quad (33a)$$

$$\text{Energy}_i(s) = E_{\text{trans},i}(s) + E_{\text{exec},i}(s) + E_{\text{enc},i}, \quad (33b)$$

$$\text{Leak}_i(s) = L_i^{\text{srv}}(s) + L_i^{\text{path}}(s) + L_i^{\text{enc}}(s) + L_i^{\text{loc}}(s). \quad (33c)$$

Using (29) and (31), we decompose $Z_i(s)$ as

$$Z_i^{\text{fac}}(s) = \alpha \sum_{(u,v) \in \mathcal{P}_i} T_{u,v}(n_{u,v}(s)) \quad (34a)$$

$$+ \beta \sum_{(u,v) \in \mathcal{P}_i} P_{u,v}(n_{u,v}(s)) \\ + \alpha G_{k_i}(n_{k_i}(s)) + \beta H_{k_i}(n_{k_i}(s)), \quad (34b)$$

$$Z_i^{\text{ind}}(s) = \alpha (D_i^{\text{prop}} + D_{\text{enc},i}) + \beta (E_i^{\text{prop}} + E_{\text{enc},i}) \\ + \gamma_{\text{privacy}} (L_i^{\text{path}}(s) + L_i^{\text{enc}}(s) + L_i^{\text{loc}}(s)), \quad (34c)$$

$$L_i^{\text{srv}}(s) = \lambda_{\text{server}} \sum_{\substack{j \neq i \\ \mu(j) = \mu(i)}} \mathbf{1}[k_j = k_i], \quad (34d)$$

$$Z_i(s) = Z_i^{\text{fac}}(s) + Z_i^{\text{ind}}(s) + \gamma_{\text{privacy}} L_i^{\text{srv}}(s). \quad (34e)$$

Each player aims to minimize $Z_i(s)$.

Definition 1 (Nash equilibrium): A joint strategy $s^* = \{s_i^*\}_{i \in \mathcal{I}}$ is a pure-strategy Nash equilibrium (NE) if, for every $i \in \mathcal{I}$ and every feasible alternative $s'_i \in \mathcal{A}_i$,

$$Z_i(s_i^*, s_{-i}^*) \leq Z_i(s'_i, s_{-i}^*),$$

where s_{-i} collects the strategies of all players other than i .

3) *Potential Game and Equilibrium*: We now construct a potential function and show that the game is an exact potential game.

Definition 2 (Ordinal and exact potential game): A finite game is an ordinal potential game if there exists a function Φ on the joint strategy space such that, for any player i and any unilateral deviation $s_i \rightarrow s'_i$,

$$Z_i(s'_i, s_{-i}) < Z_i(s_i, s_{-i}) \Rightarrow \Phi(s'_i, s_{-i}) > \Phi(s_i, s_{-i}).$$

If, in addition,

$$\Phi(s'_i, s_{-i}) - \Phi(s_i, s_{-i}) = -(Z_i(s'_i, s_{-i}) - Z_i(s_i, s_{-i}))$$

holds for all unilateral deviations, the game is said to admit an exact potential.

We define the potential in four parts. The link congestion part is

$$\begin{aligned} \Phi^{\text{link}}(s) = & -\alpha \sum_{(u,v) \in \mathcal{A}} \sum_{q=1}^{n_{u,v}(s)} T_{u,v}(q) \\ & - \beta \sum_{(u,v) \in \mathcal{A}} \sum_{q=1}^{n_{u,v}(s)} P_{u,v}(q). \end{aligned} \quad (35)$$

The node congestion part is

$$\Phi^{\text{node}}(s) = -\alpha \sum_{k=0}^N \sum_{q=1}^{n_k(s)} G_k(q) - \beta \sum_{k=0}^N \sum_{q=1}^{n_k(s)} H_k(q). \quad (36)$$

The per-task part collects the load-independent terms:

$$\Phi^{\text{ind}}(s) = - \sum_{i \in \mathcal{I}} Z_i^{\text{ind}}(s). \quad (37)$$

Finally, the server co-location part is

$$\Phi^{\text{srv}}(s) = -\gamma_{\text{privacy}} \lambda_{\text{server}} \sum_{k=0}^N \sum_{\substack{i < j \\ \mu(i) = \mu(j)}} \mathbf{1}[k_i = k_j = k]. \quad (38)$$

The total potential is

$$\Phi(s) = \Phi^{\text{link}}(s) + \Phi^{\text{node}}(s) + \Phi^{\text{ind}}(s) + \Phi^{\text{srv}}(s). \quad (39)$$

Theorem 1 (Exact potential game): Under the congestion-based representation (29), the game defined in Section IV-B2 admits $\Phi(s)$ in (39) as an exact potential. Specifically, for any player i and any unilateral deviation $s_i \rightarrow s'_i$,

$$\Phi(s'_i, s_{-i}) - \Phi(s_i, s_{-i}) = -(Z_i(s'_i, s_{-i}) - Z_i(s_i, s_{-i})). \quad (40)$$

Consequently, the game admits at least one pure-strategy Nash equilibrium, and any sequence of strict best responses terminates in a finite number of steps.

Proof: Fix a player i and a unilateral deviation from s_i to s'_i . Write $s = (s_i, s_{-i})$ and $s' = (s'_i, s_{-i})$.

(a) *Link and node congestion*: Consider an arbitrary link $(u, v) \in \mathcal{A}$. Under s , the user count is $n_{u,v}(s)$; under s' it is $n'_{u,v} = n_{u,v}(s')$. If player i does not change its usage of (u, v) , then $n'_{u,v} = n_{u,v}(s)$ and the contribution of this link to both

Φ^{link} and Z_i^{fac} is unchanged. If player i uses (u, v) under s but not under s' , then $n'_{u,v} = n_{u,v}(s) - 1$ and

$$-\alpha \sum_{q=1}^{n'_{u,v}} T_{u,v}(q) + \alpha \sum_{q=1}^{n_{u,v}(s)} T_{u,v}(q) = \alpha T_{u,v}(n_{u,v}(s)),$$

which is exactly the delay-congestion term on that link in $Z_i^{\text{fac}}(s)$. Thus the change of Φ^{link} on (u, v) equals the negative of the corresponding change in Z_i^{fac} . If player i starts using (u, v) under s' but not under s , then $n'_{u,v} = n_{u,v}(s) + 1$ and a similar calculation gives $-\alpha T_{u,v}(n'_{u,v})$, which is the negative of the new congestion cost incurred by player i on that link. The same argument applies to the energy functions $P_{u,v}(\cdot)$ and, at the node level, to $G_k(\cdot)$ and $H_k(\cdot)$ when the destination changes from k_i to k'_i . Summing over all links and destinations yields

$$\begin{aligned} \Phi^{\text{link}}(s') - \Phi^{\text{link}}(s) + \Phi^{\text{node}}(s') - \Phi^{\text{node}}(s) \\ = -(Z_i^{\text{fac}}(s') - Z_i^{\text{fac}}(s)). \end{aligned} \quad (41)$$

(b) *Per-task terms*: By definition, $\Phi^{\text{ind}}(s) = - \sum_{j \in \mathcal{I}} Z_j^{\text{ind}}(s)$. Under a unilateral deviation of player i , the quantities $Z_j^{\text{ind}}(s)$ with $j \neq i$ remain unchanged because they depend only on s_j . Hence

$$\Phi^{\text{ind}}(s') - \Phi^{\text{ind}}(s) = -(Z_i^{\text{ind}}(s') - Z_i^{\text{ind}}(s)). \quad (42)$$

(c) *Co-location leakage*: Define $I_{p,q,k}(s) \triangleq \mathbf{1}[k_p = k_q = k]$ for any tasks p, q and destination k . Then

$$I_i^{\text{srv}}(s) = \lambda_{\text{server}} \sum_{\substack{j \neq i \\ \mu(j) = \mu(i)}} \sum_{k=0}^N I_{i,j,k}(s),$$

and the co-location part of the potential is

$$\Phi^{\text{srv}}(s) = -\gamma_{\text{privacy}} \lambda_{\text{server}} \sum_{k=0}^N \sum_{\substack{p < q \\ \mu(p) = \mu(q)}} I_{p,q,k}(s).$$

Under a unilateral deviation of player i , only the indicators $I_{i,j,k}$ with $j \neq i$ may change. Using the symmetry $I_{i,j,k} = I_{j,i,k}$ and the fact that each unordered pair $\{i, j\}$ is counted once in the sum over $p < q$, we obtain

$$\begin{aligned} \Phi^{\text{srv}}(s') - \Phi^{\text{srv}}(s) \\ = -\gamma_{\text{privacy}} \lambda_{\text{server}} \sum_{\substack{j \neq i \\ \mu(j) = \mu(i)}} \sum_{k=0}^N (I_{i,j,k}(s') - I_{i,j,k}(s)) \\ = -\gamma_{\text{privacy}} (L_i^{\text{srv}}(s') - L_i^{\text{srv}}(s)). \end{aligned} \quad (43)$$

Combining (43) with (34d) and (34e) shows that the change of Φ^{srv} equals the negative of the change of the co-location part of Z_i .

(d) *Putting the pieces together*: Adding (41), (42), and (43), and using the split in (34e), we obtain

$$\begin{aligned} \Phi(s') - \Phi(s) \\ = -(Z_i^{\text{fac}}(s') - Z_i^{\text{fac}}(s)) - (Z_i^{\text{ind}}(s') - Z_i^{\text{ind}}(s)) \end{aligned}$$

Algorithm 1: PAPGS: Privacy-Aware Potential Game Scheduler.

```

1: Input: Topology  $\mathcal{A}$ , task set  $\mathcal{I}$ , resource limits (25),
   weights  $(\alpha, \beta, \gamma_{\text{privacy}})$ , privacy parameters, functions
    $T, P, G, H$ 
2: Output: Joint strategy  $s^*$ 
3: Initialize a feasible profile  $s^{(0)} = \{(k_i^{(0)}, \mathcal{P}_i^{(0)})\}_{i \in \mathcal{I}}$ 
4: Compute initial facility counts  $n_{u,v}$  and  $n_k$ 
5:  $t \leftarrow 0$ 
6: repeat
7:   Select a player  $i \in \mathcal{I}$ 
8:   Build candidate set  $\mathcal{C}_i$  by including local and
     ground-cloud execution, and for each satellite  $k \neq k_i^{(t)}$ 
     adding  $(k, \mathcal{P}_i^{(k)})$  if a path  $\mathcal{P}_i^{(k)}$  from  $s(i)$  to  $k$ 
     satisfying the hop constraint exists
9:   For each  $s'_i = (k, \mathcal{P}_i^{(k,\ell)}) \in \mathcal{C}_i$ :
     Temporarily update facility counts to obtain  $n'_{u,v}$  and
      $n'_k$ 
     Check feasibility against (25)
     Evaluate  $Z_i(s'_i, s_{-i}^{(t)})$  using (32)
10:  Choose best response  $s_i^* = \arg \min_{s'_i \in \mathcal{C}_i} Z_i(s'_i, s_{-i}^{(t)})$ 
11:  if  $Z_i(s_i^*, s_{-i}^{(t)}) < Z_i(s_i^{(t)}, s_{-i}^{(t)})$  then
12:    Commit  $s_i^{(t+1)} \leftarrow s_i^*$  and update  $n_{u,v}$  and  $n_k$ 
13:  else
14:    Set  $s_i^{(t+1)} \leftarrow s_i^{(t)}$ 
15:  end if
16:  Set  $s_j^{(t+1)} \leftarrow s_j^{(t)}$  for all  $j \neq i$ 
17:   $t \leftarrow t + 1$ 
18: until no player can strictly reduce its cost  $Z_i$ 
19: returns  $s^{(t)}$ 

```

$$\begin{aligned}
& -\gamma_{\text{privacy}}(L_i^{\text{srv}}(s') - L_i^{\text{srv}}(s)) \\
& = -(Z_i(s') - Z_i(s)),
\end{aligned}$$

which is exactly (40). Therefore, Φ is an exact potential for the game. In a finite exact potential game, at least one pure-strategy NE exists, and any sequence of strict best responses must terminate after a finite number of steps, which completes the proof.

4) *Decentralized Algorithm (PAPGS):* Based on the potential game structure, we propose a decentralized best-response procedure, termed PAPGS, that converges to a pure NE.

5) *Convergence and Complexity:* By Theorem 1, any strict best-response update that decreases some Z_i strictly increases Φ . Let $\Phi_{\min}$ be the minimum value of Φ over the finite strategy space and let $\Delta_{\min} > 0$ be the smallest strictly positive increase of Φ induced by any strict best response. Then PAPGS terminates at a pure NE in a finite number of iterations, and the number of strict-improvement iterations satisfies

$$\text{iterations} \leq \frac{\Phi(s^{(0)}) - \Phi_{\min}}{\Delta_{\min}}.$$

To make the per-iteration cost explicit, let $N_t = |\mathcal{I}|$ be the number of tasks, let $K_i \leq N + 1$ denote the number of destinations considered by task i , let P be the maximum number of candidate paths retained per destination, and let $H_{\max}$ be the routing hop bound. Suppose that candidate routes are generated on the satellite graph $G = (V, \mathcal{A})$, where $|V| = N$ and $|\mathcal{A}|$ is the number of directed ISLs. If a hop-bounded P -shortest-path search is used, then the route-generation cost for one task can be written as

$$C_{\text{route}} = O(K_i P(|\mathcal{A}| + N \log N)).$$

After the candidate routes are available, maintaining link-load counters, destination-load counters, and same-user destination counters allows feasibility checking and objective evaluation of one candidate in $O(H_{\max})$ time, because only the links on the candidate path and the destination-specific counters need to be updated. Therefore, one best-response update costs

$$O(C_{\text{route}} + K_i P H_{\max}),$$

and one full sweep over all tasks costs

$$O(N_t [C_{\text{route}} + K_i P H_{\max}]).$$

The total runtime is this per-sweep cost multiplied by the number of best-response sweeps before convergence. In the common case where only one feasible path per destination is retained, the above expression reduces to the special case $P = 1$. Since candidate generation and candidate evaluation are performed on a per-task basis, they can also be parallelized across satellites or scheduling agents.

6) *Price of Anarchy:* Let χ be the set of pure-strategy Nash equilibria and let s^{opt} be a global optimum of (28). Define the social cost

$$C(s) \triangleq \sum_{i \in \mathcal{I}} Z_i(s), \quad (44)$$

which is consistent with the centralized objective.

The co-location term charges each task by $\lambda_{\text{server}}(n_{\xi,k} - 1)$, which is affine but has a negative intercept. For PoA analysis, we use the equivalent shifted costs $\tilde{Z}_i(s) \triangleq Z_i(s) + \gamma_{\text{privacy}} \lambda_{\text{server}}$ and $\tilde{C}(s) \triangleq \sum_i \tilde{Z}_i(s) = C(s) + \gamma_{\text{privacy}} \lambda_{\text{server}} |\mathcal{I}|$. This shift does not change best responses, the NE set χ , or s^{opt} , and turns the co-location charge into a nondecreasing affine per-task facility cost in $n_{\xi,k}$ with nonnegative coefficients. Accordingly, define the PoA as

$$\text{PoA} \triangleq \frac{\max_{s \in \chi} \tilde{C}(s)}{\tilde{C}(s^{\text{opt}})} \geq 1. \quad (45)$$

Under (29)–(31), all externalities come from facilities whose cost depends on the induced loads: i) ISL links $(u, v) \in \mathcal{A}$ with load $n_{u,v}(s)$, ii) execution destinations $k \in \{0, 1, \dots, N\}$ with load $n_k(s)$, and iii) user–destination groups (ξ, k) capturing co-location with load $n_{\xi,k}(s) = |\{i : \mu(i) = \xi, k_i = k\}|$. In the operating region used for online scheduling, the fitted congestion costs $T_{u,v}(\cdot), P_{u,v}(\cdot), G_k(\cdot), H_k(\cdot)$ are well-approximated by nondecreasing affine functions with nonnegative coefficients in the operating region. We further adopt the standard convention that any infeasible action incurs an infinite (or sufficiently large)

penalty, so unilateral deviations are always well-defined; we focus on finite-cost profiles where the penalty is inactive.

Theorem 2 (PoA bound under affine congestion): If each component in $\tilde{C}(s)$ can be expressed as an affine nondecreasing per-task facility cost $c(n) = an + b$ with $a, b \geq 0$ (after incorporating the weights $\alpha, \beta, \gamma_{\text{privacy}}$ and the constant shift described above for co-location), then every pure NE $s^{\text{NE}} \in \chi$ satisfies

$$\tilde{C}(s^{\text{NE}}) \leq \frac{5}{2} \tilde{C}(s^{\text{opt}}), \quad \text{i.e.,} \quad \text{PoA} \leq \frac{5}{2}. \quad (46)$$

Proof: Fix any pure NE s and the optimum s^{opt} . By the NE condition (under the penalized/extended costs), $\tilde{Z}_i(s) \leq \tilde{Z}_i(s_i^{\text{opt}}, s_{-i})$ for all i , hence

$$\tilde{C}(s) \leq \sum_{i \in \mathcal{I}} \tilde{Z}_i(s_i^{\text{opt}}, s_{-i}). \quad (47)$$

Consider any facility with load x under s and load y under s^{opt} . When players are hypothetically switched one-by-one to their strategies in s^{opt} while others stay at s , the load seen by a deviating player on that facility is at most $x + 1$. Thus, the total facility charge on the RHS of (47) is upper bounded by $y c(x + 1)$. For affine $c(n) = an + b$ with $a, b \geq 0$, the standard smoothness inequality holds [38]:

$$y c(x + 1) \leq \frac{5}{3} y c(y) + \frac{1}{3} x c(x), \quad \forall x, y \in \mathbb{Z}_{\geq 0}. \quad (48)$$

Summing (48) over all facility components in $\tilde{C}$ (including load-independent per-task additive terms, which correspond to $a = 0$) yields

$$\sum_{i \in \mathcal{I}} \tilde{Z}_i(s_i^{\text{opt}}, s_{-i}) \leq \frac{5}{3} \tilde{C}(s^{\text{opt}}) + \frac{1}{3} \tilde{C}(s).$$

Therefore, combining with (47) gives

$$\tilde{C}(s) \leq \frac{5}{3} \tilde{C}(s^{\text{opt}}) + \frac{1}{3} \tilde{C}(s),$$

which implies $\tilde{C}(s) \leq \frac{5}{2} \tilde{C}(s^{\text{opt}})$ after rearranging.

The bound above is worst-case and typically conservative. In our experiments, we further report an empirical PoA estimate on moderate-size instances where TS-OPT returns s^{opt} , by the ratio $C(s^{\text{PAPGS}})/C(s^{\text{opt}})$, where s^{PAPGS} is the NE reached by PAPGS.

V. PERFORMANCE EVALUATION

This section presents a numerical evaluation of the proposed privacy-aware scheduling framework. We first describe the simulation setup and parameter configurations. We then compare the proposed methods with representative learning-based and heuristic baselines under a unified objective that jointly accounts for latency, energy consumption, and privacy leakage. The evaluation is organized in two parts: i) small-scale experiments that validate solution quality against an optimal reference, and ii) large-scale experiments that assess scalability, robustness under heavy workloads, and privacy preservation in heterogeneous LEO constellations. Unless otherwise specified, we report the overall system cost in (28a) together with end-to-end delay, energy consumption, privacy leakage, task completion rate, and per-instance runtime.

TABLE II
PARAMETER SETTINGS

Parameter	Symbol / Unit	Value
ISL hop budget	$H_{\max}$	2
Compute capacity per sat	SC_n (cycles s^{-1})	1.0×10^{10}
Ground cloud capacity	GC (cycles s^{-1})	1.0×10^{13}
ISL carrier frequency	f_{ISL} (GHz)	40
ISL bandwidth	B_{ISL} (MHz)	500
Ground-satellite carrier	f_{GS} (GHz)	20
Link margin	M (dB)	6
Noise temperature	T_s (K)	300
Boltzmann constant	k_B (J/K)	1.38×10^{-23}
Earth radius	R_{Earth} (km)	6371
Speed of light	c (m/s)	3.0×10^8
Task deadline	t_i (s)	$\mathcal{U}[1, 11]$
CPU cycles per bit	c_i (cycles/bit)	$\mathcal{U}[200, 400]$
Encryption ratio	$\Pr(e_i = 1)$	0.5
Trust weight	$\lambda_{\text{loc}}(k)$	1.0 (ground), 0.6 (satellite)
Satellite TX power	P_{sat} (dBm)	50 (100 W)
User TX power	P_{usr} (dBm)	30 (1 W)
Energy coeff	ε	1.0×10^{-27}
Delay weight	α	100
Energy weight	β	1.0×10^{-4}
Privacy weight	γ_{privacy}	10

A. Experimental Settings

1) *Simulation Platform:* The simulation is conducted on a workstation with an Intel i5-14600KF CPU, 32 GB DDR5 RAM, and an NVIDIA RTX 4080 SUPER GPU. We implemented the environment in Python 3.9, using PyTorch 2.7.0 for the DRL-based SAC baseline and NumPy 1.26 for game-theoretic interactions. For each configuration, we run independent Monte Carlo trials with different random seeds and report the averaged metrics. For reproducibility, TS-OPT is solved in Gurobi with MIPGap=0, MIPGapAbs=0, Threads=4, Presolve=2, Cuts=2, Heuristics=0.05, MIPFocus=1, and ConcurrentMIP=2. No explicit time limit is imposed unless otherwise stated. Table II summarizes the key physical-layer, computing, and privacy-related parameters used throughout the simulations, unless otherwise specified [39], [40], [41].

2) *Task and Routing Configuration:* For reproducibility, we explicitly specify the task-generation and routing-related settings used in the simulations. The execution-location trust weight adopts a destination-class assignment motivated by prior privacy-aware edge/satellite computing studies [6], [14], the ground cloud is assigned a higher trust weight, i.e., $\lambda_{\text{loc}}(0) = 1.0$, whereas all satellite execution destinations use a uniform trust weight $\lambda_{\text{loc}}(k) = 0.6$ for $k \in \{1, \dots, N\}$. This setting reflects the common assumption that the ground cloud is relatively more trusted and better protected than opportunistically selected in-orbit execution nodes, while also keeping the privacy term transparent and reproducible across Monte Carlo runs. Task sizes are controlled by the scenario setting rather than sampled independently: the small-scale study uses task sizes in $\{150, 200, 250, 300, 350, 400\}$ KB, whereas the large-scale study uses $\{0.5, 1.0, 1.5, 2.0, 2.5, 3.0\}$ MB. For each task, the CPU-cycle density is drawn i.i.d. from $\mathcal{U}[200, 400]$ cycles/bit, the deadline is drawn i.i.d. from $\mathcal{U}[1, 11]$ s, and the encryption

indicator is sampled independently from a Bernoulli distribution with parameter 0.5. Unless otherwise stated, all reported results are averaged over 50 independent Monte Carlo runs per configuration.

For PAPGS, the candidate set of each task includes local execution, direct ground-cloud offloading when a feasible contact exists, and remote-satellite execution over ISL paths. In the simulations, the ISL hop budget is set to $H_{\max} = 2$, so that PAPGS searches only short multi-hop relay paths. This choice is consistent with delay-sensitive SEC scheduling, since longer relay chains enlarge transmission delay, queueing risk, and exposure surface, and are therefore unlikely to remain attractive under the unified objective. For each reachable remote destination, we retain one minimum-hop feasible path, and the initial PAPGS profile is set to all-local execution. This configuration keeps online candidate generation lightweight while still preserving the main placement and routing trade-off in intra-constellation scheduling.

Consistently with (17), hop count in the simulations counts only ISL hops. Therefore, local execution and direct ground-cloud offloading both have zero ISL hops, whereas remote-satellite execution is measured by the number of traversed ISL segments.

3) *Weight Selection Guideline*: Because delay, energy, and privacy leakage have different numerical scales, the weights in (28a) should be selected after a short calibration step rather than directly from raw magnitudes. Let $\bar{D}$, $\bar{E}$, and $\bar{L}$ denote the average unweighted total delay, energy, and leakage measured on a small representative set of pilot instances. An equal-contribution initialization can be obtained by setting

$$(\alpha, \beta, \gamma_{\text{privacy}}) \propto \left(\frac{1}{\bar{D}}, \frac{1}{\bar{E}}, \frac{1}{\bar{L}} \right),$$

followed by a common rescaling for numerical convenience. This initialization makes the three weighted components comparable in magnitude in the unified objective. Mission preferences can then be incorporated by increasing the corresponding weight: a larger α emphasizes delay-sensitive services, a larger β prioritizes energy-limited operation, and a larger γ_{privacy} favors privacy-preserving routing and execution choices. Unless otherwise stated, the experiments use $\alpha = 100$, $\beta = 1.0 \times 10^{-4}$, and $\gamma_{\text{privacy}} = 10$, chosen according to this calibration-and-tuning procedure.

4) *Baselines and Metrics*: We evaluate two complementary schedulers proposed in this work, together with five representative baselines.

- *TS-OPT (Optimal)*: A centralized solver-based optimizer that solves (28) to global optimality for small-to-moderate instances. We use TS-OPT as an optimal reference to validate solution quality and quantify the optimality gap. Due to its rapidly increasing runtime with network scale and load, TS-OPT is primarily suited for delay-tolerant missions where scheduling can be performed offline (e.g., batch analytics and non-urgent planning).
- *PAPGS (Real-Time Scheduler)*: The Privacy-Aware Potential Game Scheduler in Section IV-B. PAPGS performs decentralized best-response updates and converges to a

pure-strategy Nash equilibrium with low overhead, targeting latency-sensitive SEC operations that require fast online decisions (e.g., time-critical monitoring and emergency response).

- *MFGLB [26]*: A mean-field-game-based task-offloaded load-balancing baseline originally designed for large-scale MEC systems. In our experiments, it is adapted to the SEC setting under the same routing and feasibility constraints.
- *GDCO [42]*: A game-theoretic distributed computation offloading baseline for LEO satellite-terrestrial edge computing systems. It computes offloading decisions through game-theoretic strategy updates toward a Nash equilibrium.
- *SAC [9]*: A learning-based baseline built upon the Soft Actor-Critic algorithm. It models task scheduling as a constrained Markov decision process and learns offloading policies via entropy-regularized policy updates.
- *Local*: Executes each task at its source satellite whenever feasible, without inter-satellite forwarding or ground offloading.
- *Random*: Assigns each task uniformly at random among all reachable destinations (satellite or ground).

We use the overall system cost in (28a) as the primary metric. Because this quantity is a weighted sum of heterogeneous components, it is reported as a dimensionless unified cost score and is used for relative comparison under the same weight setting. For the ablation study where γ_{privacy} is changed, the cross-setting comparison mainly relies on delay, completion rate, and privacy leakage score, while the unified cost is reported for completeness under each setting's own objective. Additionally, this section reports end-to-end delay, total energy consumption, the privacy leakage score in (22), the completion rate (defined as the fraction of tasks meeting the deadline constraint), and per-instance runtime.

B. Ablation on the Privacy Term

To clarify whether PAPGS remains effective on basic scheduling performance even without explicitly optimizing privacy, we conduct an ablation study by rerunning PAPGS with the privacy weight changed from $\gamma_{\text{privacy}} = 10$ to $\gamma_{\text{privacy}} = 0$. In this ablation, all other settings are kept unchanged, including the task-generation protocol, trust-weight assignment, routing candidate generation, hop budget, initialization rule, and stopping criterion. Since the unified cost is evaluated under each setting's own objective, the cross-setting comparison mainly focuses on delay, completion rate, and privacy leakage score.

Table IV reports both aggregate and representative-scenario results under the current repository aggregation protocol. Specifically, the small-scale average is computed over the repository's current small-scale aggregate setting, while the large-scale average is computed over the enlarged large-scale evaluation range with $N \in \{15, 21, 27\}$, payload sizes in $\{1.0, 1.5, 2.0, 2.5, 3.0\}$ MB, and task counts in $\{50, 60, 70, 80, 90, 100\}$. We also include two representative large-scale scenarios, namely $N = 21$ with 1.5 MB payloads

TABLE IV
ABLATION STUDY OF PAPGS WITH AND WITHOUT THE PRIVACY TERM

Scenario	Setting	Unified Cost	Delay (s)	Completion Rate (%)	Privacy Leakage Score	Runtime (s)
Small-scale avg.	PAPGS ($\gamma_{\text{privacy}} = 10$)	31.46	0.142	100.000	1.387	0.011
	PAPGS w/o privacy ($\gamma_{\text{privacy}} = 0$)	17.12	0.135	100.000	1.552	0.011
Large-scale avg.	PAPGS ($\gamma_{\text{privacy}} = 10$)	145.84	0.929	98.254	1.807	0.307
	PAPGS w/o privacy ($\gamma_{\text{privacy}} = 0$)	127.04	0.926	98.262	2.091	0.258
21 Sat, 1.5 MB	PAPGS ($\gamma_{\text{privacy}} = 10$)	111.46	0.702	99.767	1.764	0.265
	PAPGS w/o privacy ($\gamma_{\text{privacy}} = 0$)	93.02	0.695	99.787	2.072	0.227
27 Sat, 2.5 MB	PAPGS ($\gamma_{\text{privacy}} = 10$)	166.68	1.071	98.216	1.792	0.553
	PAPGS w/o privacy ($\gamma_{\text{privacy}} = 0$)	148.01	1.077	98.281	2.094	0.450

and $N = 27$ with 2.5 MB payloads, to provide a more direct view of the privacy–performance trade-off.

The results show that PAPGS remains competitive on basic scheduling performance even without the privacy term, while the privacy-aware setting achieves a clearly lower privacy leakage score with only a minor impact on delay and completion rate. On the small-scale aggregate setting, enabling the privacy term reduces the privacy leakage score from 1.552 to 1.387, i.e., by about 10.7%, while keeping the completion rate unchanged at 100%; the average delay increases only slightly from 0.135 s to 0.142 s. On the large-scale aggregate setting, the privacy-aware PAPGS lowers the privacy leakage score from 2.091 to 1.807, i.e., by about 13.6%, whereas the average delay changes only marginally from 0.926 s to 0.929 s and the completion rate remains essentially unchanged (98.262% versus 98.254%).

A similar pattern is observed in the representative large-scale scenarios. For $N = 21$ and 1.5 MB payloads, enabling the privacy term reduces the privacy leakage score from 2.072 to 1.764 (about 14.9%) with only a minor delay increase from 0.695 s to 0.702 s and a negligible completion-rate change. For $N = 27$ and 2.5 MB payloads, the privacy leakage score decreases from 2.094 to 1.792 (about 14.4%), while the delay remains nearly unchanged and is even slightly lower in the privacy-aware run (1.071 s versus 1.077 s). Overall, these results indicate that the advantage of PAPGS is not solely driven by explicitly adding the privacy term to the objective. Instead, PAPGS preserves strong basic scheduling capability and, at the same time, provides an effective and controllable privacy–performance trade-off.

C. Optimality and Runtime Analysis (Small-Scale)

We first examine the small-scale regime, where the optimal solution to (28) remains computationally attainable and can therefore serve as a reliable reference for validating solution quality. We consider $N = 6 \sim 9$ satellites and vary the task population from 6 to 36. This setting is selected because the decision space already couples routing with execution placement, while TS-OPT can still return the global optimum without timeouts. This enables a direct evaluation of how close PAPGS operates to the optimum and how much decision overhead it incurs in practice. Table III summarizes the averages over 50 Monte Carlo runs.

Table III indicates that PAPGS remains the closest method to the optimal reference TS-OPT in the small-scale regime. The unified cost of PAPGS is 39.738 compared with 39.667

TABLE III
AVERAGE PERFORMANCE COMPARISON (SMALL-SCALE SCENARIOS)

Algorithm	Unified Cost	Delay (s)	Gap to Opt.	Runtime (s)
TS-OPT	39.667	0.218	-	0.569
PAPGS	39.738	0.216	0.18%	0.009
MFGLB	59.296	0.395	49.48%	0.191
GDCO	78.250	0.611	97.27%	0.113
SAC	161.548	1.432	307.26%	0.042
Local	294.809	2.843	643.21%	0.000

Note: “Unified Cost” denotes the objective value in (28a). Since it is a weighted sum of heterogeneous terms, it is reported as a dimensionless score and is used only for relative comparison under the same weight setting.

for TS-OPT, corresponding to only a 0.18% gap, while the average delay is also nearly identical (0.216 s versus 0.218 s). By comparison, MFGLB and GDCO yield unified costs of 59.296 and 78.250, i.e., 49.48% and 97.27% above TS-OPT, with average delays of 0.395 s and 0.611 s, respectively. SAC and Local remain much worse in both cost and delay. This near-optimality is consistent with the potential-game design, where each best-response update directly reduces the aligned system objective and thus quickly settles at a high-quality equilibrium in a still-manageable decision space. The difference mainly appears in scheduling overhead, where PAPGS takes only 0.009 s per instance, whereas TS-OPT requires 0.569 s because of the combinatorial burden of centralized search. PAPGS is also substantially faster than MFGLB and GDCO, whose runtimes are 0.191 s for MFGLB and 0.113 s for GDCO. Concretely, each PAPGS best-response step evaluates link congestion, destination congestion, hop-dependent exposure, and destination trust under the same weighted objective. This mechanism keeps the decentralized decisions close to the low-cost placement-and-routing structure selected by TS-OPT in small solvable instances.

Fig. 2 further confirms the small-scale observation with representative values. For $N = 6$ and 200 KB in Fig. 2(a), PAPGS nearly overlaps TS-OPT across the entire sweep and stays in the same low-cost band (about 33.9–38.2 versus 33.8–38.1 for TS-OPT). MFGLB remains around 44.9–57.2 and GDCO around 44.9–56.2, both clearly better than SAC but still noticeably above PAPGS and TS-OPT. SAC stays above 117.9 throughout, while Local exceeds 197.8. For $N = 9$ and 200 KB in Fig. 2(b), PAPGS again tracks TS-OPT closely in a narrow 28–31 cost band. MFGLB stays in a moderate range of 38.3–50.3, whereas

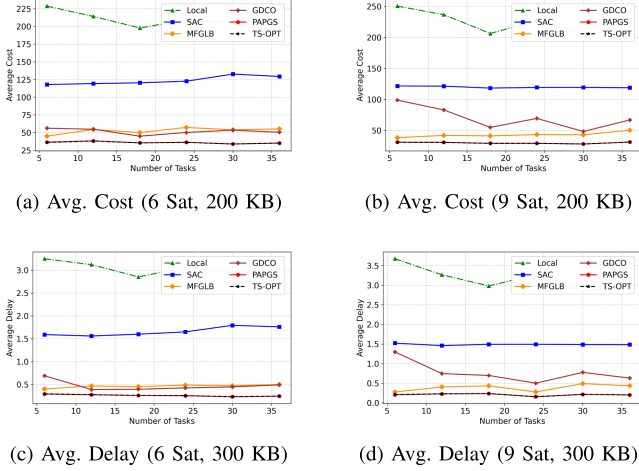


Fig. 2. Performance comparison in representative small-scale networks.

GDCO becomes less stable and varies from 48.3 up to 98.8, especially under lighter loads. SAC remains above 118 and Local above 206, maintaining a persistent multi-fold gap from PAPGS. These results confirm that PAPGS remains the closest scalable scheduler to TS-OPT in representative small-scale instances, whereas MFGLB and GDCO, although substantially stronger than SAC and heuristic methods, still leave a visible gap in solution quality.

The same trend appears in delay. Under 300 KB in Fig. 2(c), PAPGS stays very close to TS-OPT at about 0.230–0.289 s, while MFGLB increases to 0.400–0.498 s and GDCO to 0.387–0.689 s. SAC remains much higher at 1.558–1.792 s, and Local is further slowed by queueing at the access satellite, reaching 2.852–3.247 s. A similar pattern is observed in Fig. 2(d), which shows that PAPGS and TS-OPT remain in the same low-delay band (about 0.154–0.235 s), MFGLB rises to 0.276–0.489 s, and GDCO varies more widely between 0.501 s and 1.298 s. Random is orders of magnitude worse because uninformed assignments frequently incur long waiting and deadline violations, which explains the clear separation observed in both delay and cost.

D. Scalability Analysis: Impact of Satellite Density

This set of experiments examines how constellation density affects scheduling performance when the overall workload increases. We vary the number of satellites as $N \in \{15, 21, 27\}$ while keeping the other settings unchanged, and focus on regimes where a real-time scheduler is required. For these large-scale instances, the centralized TS-OPT is not plotted, since its solver runtime is already impractical. As satellite density increases, we compare PAPGS with all scalable baselines under the same SEC setting.

1) *Delay and Task Completion Rate*: Fig. 3 reports end-to-end delay at a moderate load of 1.5 MB (Fig. 3(a)–(c)) and task completion rate at a heavier load of 2.5 MB (Fig. 3(d)–(f)), with the number of tasks increasing from 50 to 100 in all cases.

In Fig. 3(a)–(c), PAPGS consistently achieves the lowest delay among all scalable baselines as satellite density increases. For $N = 15$ and 1.5 MB payloads, PAPGS keeps delay within

TABLE V
AVERAGE PRIVACY LEAKAGE SCORE AT 1.5 MB OVER TASK LOADS FROM 50 TO 100

N	PAPGS (Avg.)	MFGLB (Avg.)	GDCO (Avg.)	SAC (Avg.)
15	1.837	1.663	1.573	2.098
21	1.737	1.612	1.523	2.252
27	1.689	1.621	1.560	2.282

0.748–0.852 s, whereas MFGLB and GDCO remain clearly higher at 1.602–2.033 s and 1.365–1.855 s, respectively. When the constellation grows to $N = 21$, PAPGS further stays in a low-delay range of 0.652–0.760 s, while MFGLB and GDCO remain at 1.426–1.938 s and 1.236–1.799 s. With $N = 27$, PAPGS reduces delay to 0.633–0.683 s and continues to outperform MFGLB and GDCO. Specifically, MFGLB stays at 1.296–1.643 s, whereas GDCO increases to 1.740–2.199 s. In contrast, SAC remains above 5 s even at $N = 27$, and Local stays around 12.7–13.2 s. These results show that PAPGS consistently achieves the lowest delay among the compared scalable schedulers as constellation density increases.

The second row of Fig. 3 reports task completion rate at 2.5 MB, which is consistent with the delay trends above. For $N = 15$, PAPGS maintains 95.20%–97.34% completion, compared with 77.84%–83.40% for MFGLB and 77.02%–85.50% for GDCO. For $N = 21$, PAPGS remains at 96.56%–98.00%, while MFGLB stays at 82.80%–84.95% and GDCO at 85.08%–88.16%. For $N = 27$, PAPGS still achieves 97.17%–98.20%, whereas MFGLB reaches 83.74%–88.00% and GDCO varies more widely between 74.76% and 83.29%. SAC remains much lower, at about 57.71%–58.73% for $N = 15$, 33.60%–35.40% for $N = 21$, and 78.51%–79.08% for $N = 27$. Overall, PAPGS still achieves the highest completion rate under heavy workloads across the compared scalable schedulers.

2) *Privacy Preservation*: Table V summarizes the average privacy leakage at 1.5 MB over task loads from 50 to 100. Compared with SAC, PAPGS consistently achieves a lower privacy leakage score across all constellation sizes, reducing the average score from 2.098 to 1.837 at $N = 15$, from 2.252 to 1.737 at $N = 21$, and from 2.282 to 1.689 at $N = 27$. This corresponds to relative reductions of about 12.4%, 22.9%, and 26.0%, respectively. MFGLB and GDCO further obtain lower privacy leakage scores than PAPGS, with MFGLB at 1.663/1.612/1.621 and GDCO at 1.573/1.523/1.560 for $N = 15/21/27$, respectively. This behavior is consistent with their tendency to adopt more conservative routing and execution choices, which can reduce privacy exposure but also leads to higher delay and lower completion rate in the large-scale experiments. Therefore, the privacy results should be interpreted together with delay, completion rate, and runtime rather than in isolation. Under this broader view, PAPGS delivers the most balanced overall operating point across the compared schedulers under the unified SEC objective.

Local can exhibit a lower privacy leakage score because it does not perform task scheduling, which greatly reduces the decision space for routing and execution placement and therefore

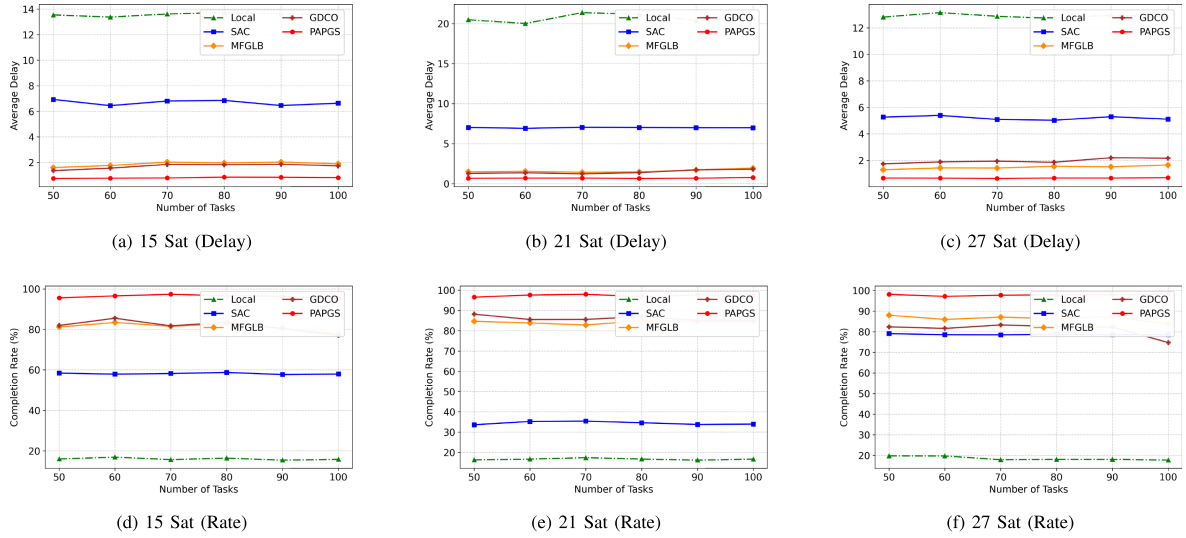


Fig. 3. Performance metrics under varying satellite densities. (a)–(c) Average delay at 1.5 MB; (d)–(f) task completion rate at 2.5 MB.

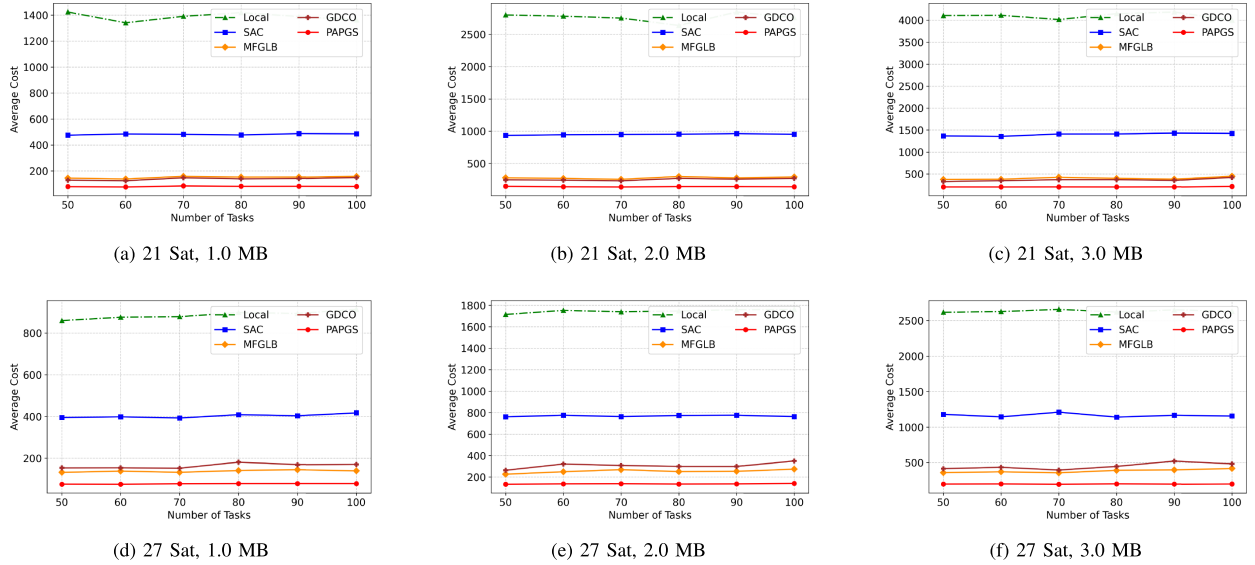


Fig. 4. System cost evolution under increasing data load.

naturally lowers exposure to privacy risks. This lower leakage comes together with substantially worse delay and completion performance, as shown in Fig. 3.

E. Scalability Under Increasing Load and Summary

We stress-test the schedulers under heavy data loads, where transmission time, encryption overhead, and queuing pressure increase simultaneously. Fig. 4 shows how the objective value changes as the number of tasks increases from 50 to 100 under two dense constellations ($N = 21$ and $N = 27$) and three data sizes (1.0, 2.0, and 3.0 MB). Although the plotted metric is the unified cost in (28a), its variation in this regime is dominated mainly by end-to-end delay growth and the resulting deadline violations.

Fig. 4 shows that heavier payloads and more tasks increase cost mainly by amplifying transmission time and queuing pressure, which in turn increases deadline misses under contention. For $N = 21$, PAPGS grows from 76.028–84.559 at 1.0 MB to 203.112–217.139 at 3.0 MB, and remains the lowest-cost method throughout. The two game-theoretic baselines form an intermediate tier, where GDCO is generally lower than MFLB in this 21-satellite setting, rising from 123.773–148.715 at 1.0 MB to 325.215–421.507 at 3.0 MB, while MFLB increases from 137.626–158.213 to 373.834–449.151 over the same load range. SAC remains much higher, from 475.650–487.787 at 1.0 MB to 1356.118–1429.614 at 3.0 MB, and Local increases even faster, reflecting the inability to shift workload away from congested access satellites as the system enters a queue-dominated regime.

TABLE VI
OVERALL AVERAGE PERFORMANCE (LARGE-SCALE SCENARIOS)

Algorithm	Delay (s)	Completion Rate (%)	Privacy Score	Runtime (s)
PAPGS	0.947	98.094	1.789	0.284
MFGLB	2.164	87.797	1.628	1.776
GDCO	2.244	86.902	1.552	11.547
SAC	8.423	62.549	2.210	0.154
Local	20.985	24.494	1.497	0.001
Random	3.805×10^6	19.712	1.385	0.001

Note: “privacy score” denotes the privacy leakage score.

For $N = 27$, the overall cost level decreases across all data sizes, consistent with richer offloading options enabled by a denser constellation. PAPGS remains the lowest-cost method, staying at 75.070–78.967 for 1.0 MB, 132.471–141.418 for 2.0 MB, and 194.534–201.435 for 3.0 MB. In this denser setting, MFGLB becomes consistently better than GDCO, with cost ranges of 132.365–145.116, 226.270–274.365, and 356.552–418.700 from 1.0 MB to 3.0 MB, whereas GDCO rises to 152.137–181.013, 263.112–350.333, and 395.406–521.752. SAC also benefits from higher density but still remains several times above PAPGS, indicating limited robustness to heavy-load congestion under the coupled routing-and-placement decisions.

The advantage of PAPGS becomes more pronounced under heavy workloads, where the unified objective is increasingly dominated by queueing delay and multi-hop forwarding overhead. In this regime, each best-response update can react directly to local congestion and path conditions, so PAPGS more effectively steers tasks away from overloaded relays and preserves shorter end-to-end execution paths. This behavior explains why the cost gap widens as payload size and task volume increase under congested SEC conditions.

Table VI aggregates the large-scale experiments in this section and in the density study (satellite counts $N \in \{15, 21, 27\}$ and data sizes in $\{1.0, 1.5, 2.0, 2.5, 3.0\}$ MB). PAPGS provides the most balanced operating point, with 0.947 s average delay, 98.094% completion rate, privacy leakage score 1.789, and runtime 0.284 s. MFGLB reaches 2.164 s delay, 87.797% completion rate, privacy leakage score 1.628, and runtime 1.776 s, while GDCO yields 2.244 s delay, 86.902% completion rate, privacy leakage score 1.552, and runtime 11.547 s. Their lower privacy leakage scores suggest that they more often select conservative scheduling decisions that reduce exposure, but this comes at the cost of noticeably higher delay, lower completion rate, and, especially for GDCO, much larger runtime. SAC is faster than GDCO but operates at 8.423 s delay and only 62.549% completion rate with a higher privacy leakage score of 2.210, making it less suitable under strict deadlines. Local and Random show lower privacy leakage scores (1.497 and 1.385), but at the cost of very high latency (20.985 s and 3.805×10^6 s) and low completion rates (24.494% and 19.712%), so they are not competitive for delay-sensitive missions.

Fig. 5 illustrates why TS-OPT is used only as a small-scale reference. Its runtime increases from 0.329 s at $N = 6$ to 2.265

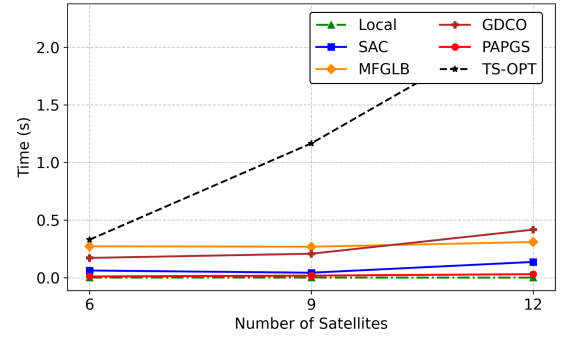


Fig. 5. Runtime scalability with respect to satellite count. PAPGS remains sub-second per instance at large scale, while TS-OPT becomes increasingly expensive as the search space grows.

s at $N = 12$, reflecting the rapidly expanding combinatorial search space of joint routing and placement. In contrast, PAPGS evaluates only local alternatives per update and remains within 0.010–0.030 s in the same regime, which makes it suitable for online scheduling. MFGLB and GDCO are both noticeably slower than PAPGS, with MFGLB staying at about 0.268–0.309 s and GDCO growing from 0.171 s at $N = 6$ to 0.416 s at $N = 12$. Therefore, PAPGS remains the most runtime-efficient scheduler among the compared decentralized methods in the tested regime.

VI. CONCLUSION

This paper studied privacy-aware task scheduling in SEC by jointly optimizing execution placement and multi-hop forwarding decisions under heterogeneous computing and communication resources. We formulated a unified optimization objective that jointly balances latency, energy, and task-level privacy leakage, with leakage capturing co-location risk, path exposure, encryption-dependent vulnerability, and destination trust heterogeneity. Based on this formulation, we developed TS-OPT as a centralized optimal reference for solvable instances and proposed PAPGS as a scalable decentralized scheduler by casting the problem into an exact potential game with finite-step convergence under strict best responses. Simulations demonstrated that PAPGS attains near-optimal performance in small-scale settings and consistently outperforms learning-based and heuristic baselines at large scale, while remaining robust under heavy workloads.

Future work will consider richer time-varying trust and adversary models and tighter integration with predictive topology/contact dynamics, since trust and attack surfaces may evolve, and foreseeable contact changes can be exploited to avoid unnecessary waiting and exposure.

REFERENCES

- [1] L. Zhong, Y. Li, M.-F. Ge, M. Feng, and S. Mao, “Joint task offloading and resource allocation for LEO satellite-based mobile edge computing systems with heterogeneous task demands,” *IEEE Trans. Veh. Technol.*, vol. 74, no. 7, pp. 11337–11352, Jul. 2025.
- [2] C. Guimarães, A. Netti, M. Sauer, F. Zeiger, H.-P. Huth, and E. Boriskova, “A survey on satellite computing: Connecting the dots between networks and applications,” *IEEE Commun. Surv. Tuts.*, vol. 28, pp. 567–592, 2025.

- [3] Y. Zengshan et al., "A comprehensive survey of orbital edge computing: Systems, applications, and algorithms," *Chin. J. Aeronaut.*, vol. 38, no. 7, 2025, Art. no. 103316.
- [4] Z. Lai et al., "LeoCC: Making internet congestion control robust to leo satellite dynamics," in *Proc. ACM SIGCOMM 2025 Conf.*, 2025, pp. 129–146.
- [5] Y. Deng et al., "Time-varying bottleneck links in LEO satellite networks: Identification, exploits, and countermeasures," in *Proc. Annu. Netw. Distrib. Syst. Secur. Symp.*, 2025, pp. 1–17.
- [6] W. Lan, K. Chen, Y. Li, J. Cao, and Y. Sahni, "Deep reinforcement learning for privacy-preserving task offloading in integrated satellite-terrestrial networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 10, pp. 9678–9691, Oct. 2024.
- [7] J. Peng et al., "Security enhanced computation offloading for collaborative inference at semantic-communication-empowered edge," *IEEE Trans. Mobile Comput.*, vol. 24, no. 9, pp. 8071–8088, Sep. 2025.
- [8] W. Fan, Q. Meng, G. Wang, H. Bian, Y. Liu, and Y. Liu, "Satellite edge intelligence: DRL-based resource management for task inference in LEO-based satellite-ground collaborative networks," *IEEE Trans. Mobile Comput.*, vol. 24, no. 10, pp. 10710–10728, Oct. 2025.
- [9] Q. Tang et al., "Joint service deployment and task scheduling for satellite edge computing: A two-timescale hierarchical approach," *IEEE J. Sel. Areas Commun.*, vol. 42, no. 5, pp. 1063–1079, May 2024.
- [10] T. Huang, Z. Fang, Q. Tang, R. Xie, T. Chen, and F. R. Yu, "Dual-timescales optimization of task scheduling and resource slicing in satellite-terrestrial edge computing networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 12, pp. 14111–14126, Dec. 2024.
- [11] R. Zhang, Y. Feng, Y. Yang, X. Li, and H. Li, "Learning-based deadlock-free multi-objective task offloading in satellite edge computing with data-dependent constraints and limited buffers," *IEEE Trans. Netw. Sci. Eng.*, vol. 12, no. 1, pp. 356–368, Jan./Feb. 2025.
- [12] Z. Wang et al., "Location privacy-aware task offloading in mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 23, no. 3, pp. 2269–2283, Mar. 2024.
- [13] J. Li et al., "Revisiting location privacy in MEC-enabled computation offloading," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 4396–4407, 2025.
- [14] W. Lan et al., "Security-sensitive task offloading in integrated satellite-terrestrial networks," *IEEE Trans. Mobile Comput.*, vol. 24, no. 3, pp. 2220–2233, Mar. 2025.
- [15] S. Chen and X. Wang, "PtrTasking: Pointer network based task scheduling for multi-connectivity enabled MEC services," *IEEE Trans. Mobile Comput.*, vol. 23, no. 10, pp. 9398–9409, Oct. 2024.
- [16] A. Du, J. Jia, S. Dustdar, J. Chen, and X. Wang, "Online service placement, task scheduling, and resource allocation in hierarchical collaborative MEC systems," *IEEE Trans. Services Comput.*, vol. 18, no. 2, pp. 983–997, Mar./Apr. 2025.
- [17] X. Xu, Y. Hu, G. Cui, L. Qi, W. Dou, and Z. Cai, "CADEC: A combinatorial auction for dynamic distributed DNN inference scheduling in edge-cloud networks," *IEEE Trans. Mobile Comput.*, vol. 24, no. 10, pp. 10024–10041, Oct. 2025.
- [18] K. Fan, B. Feng, X. Zhang, and Q. Zhang, "Demand-driven task scheduling and resource allocation in space-air-ground integrated network: A deep reinforcement learning approach," *IEEE Trans. Wireless Commun.*, vol. 23, no. 10, pp. 13053–13067, Oct. 2024.
- [19] S. Yao et al., "LEOEdge: A satellite-ground cooperation platform for the AI inference in large LEO constellation," *IEEE J. Sel. Areas Commun.*, vol. 43, no. 1, pp. 36–50, Jan. 2025.
- [20] Y. Li et al., "Stable hierarchical routing for operational LEO networks," in *Proc. 30th Annu. Int. Conf. Mobile Comput. Netw.*, 2024, pp. 296–311.
- [21] H. Wu, Y. Han, M. Rajpal, Q. Zhang, and J. Wang, "SaTE: Low-latency traffic engineering for satellite networks," in *Proc. ACM SIGCOMM 2025 Conf.*, 2025, pp. 896–916.
- [22] S. Zhang, X. Li, and K. L. Yeung, "Segment routing for traffic engineering and effective recovery in low-earth orbit satellite constellations," *Digit. Commun. Netw.*, vol. 10, no. 3, pp. 706–715, 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2352864822001973>
- [23] S. Theeranantachai, B. Zhang, and L. Zhang, "NDN's stateful forwarding plane in the presence of ground-satellite handovers," in *Proc. IEEE 32nd Int. Conf. Netw. Protoc.*, 2024, pp. 1–11.
- [24] Y. Chen et al., "Unraveling physical space limits for leo network scalability," in *Proc. 23rd ACM Workshop Hot Topics Netw.*, New York, NY, USA: Association for Computing Machinery, 2024, pp. 43–51, doi: [10.1145/3696348.3696885](https://doi.org/10.1145/3696348.3696885).
- [25] Y. Li et al., "Small-scale LEO satellite networking for global-scale demands," in *Proc. ACM SIGCOMM 2025 Conf.*, New York, NY, USA: Association for Computing Machinery, 2025, pp. 917–931, doi: [10.1145/3718958.3750525](https://doi.org/10.1145/3718958.3750525).
- [26] G. Wu, H. Wang, H. Zhang, Y. Shen, S. Shen, and S. Yu, "Mean-field game-based task-offloaded load balance for industrial mobile edge computing systems using software-defined networking," *IEEE Trans. Mobile Comput.*, vol. 23, no. 12, pp. 13773–13786, Dec. 2024.
- [27] Y. Wang, C. Yang, T. Li, X. Mi, L. Li, and Z. Han, "A survey on mean-field game for dynamic management and control in space-air-ground network," *IEEE Commun. Surv. Tut.*, vol. 26, no. 4, pp. 2798–2835, Fourth Quarter 2024.
- [28] Z. Ji, S. Wu, and C. Jiang, "Cooperative multi-agent deep reinforcement learning for computation offloading in digital twin satellite edge networks," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 11, pp. 3414–3429, Nov. 2023.
- [29] C.-Y. Hsieh, Y. Ren, and J.-C. Chen, "Edge-cloud offloading: Knapsack potential game in 5G multi-access edge computing," *IEEE Trans. Wireless Commun.*, vol. 22, no. 11, pp. 7158–7171, Nov. 2023.
- [30] J. Xu et al., "A privacy-preserving auction for task offloading and resource allocation in UAV-assisted MEC," *IEEE Trans. Mobile Comput.*, vol. 25, no. 2, pp. 2611–2626, Feb. 2026.
- [31] D. Li et al., "CPAKA: Mutual authentication and key agreement scheme based on conditional PUF in space-air-ground integrated network," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 4, pp. 3487–3500, Jul./Aug. 2023.
- [32] Z. Seyed, F. Rahmati, M. Ali, and X. Liu, "A fully decentralized auditing approach for edge computing: A game-theoretic perspective," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 6, pp. 6121–6132, Nov./Dec. 2025.
- [33] M. Huang, Z. Li, A. Liu, X. Zhang, Z. Yang, and M. Yang, "A proactive trust evaluation system for secure data collection based on sequence extraction," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 2, pp. 997–1010, Mar./Apr. 2025.
- [34] X. Xu, K. Meng, H. Xiang, G. Cui, X. Xia, and W. Dou, "Blockchain-enabled secure, fair and scalable data sharing in zero-trust edge-end environment," *IEEE J. Sel. Areas Commun.*, vol. 43, no. 6, pp. 2056–2069, Jun. 2025.
- [35] X. Tang, M. Shen, Q. Li, L. Zhu, T. Xue, and Q. Qu, "PILE: Robust privacy-preserving federated learning via verifiable perturbations," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 6, pp. 5005–5023, Nov./Dec. 2023.
- [36] 3GPP, "Study on new radio (NR) to support non-terrestrial networks," 3rd Generation Partnership Project (3GPP), Tech. Rep. (TR), TR 38.811, version 15.4.0, Release 15, Oct. 2020.
- [37] H. M. Al-Gunid et al., "NOMA-enabled integrated space-ground cellular networks architecture relying on control-and-user-plane separation," *IEEE J. Sel. Areas Commun.*, vol. 42, no. 12, pp. 3690–3704, Dec. 2024.
- [38] T. Roughgarden, "Intrinsic robustness of the price of anarchy," *Commun. ACM*, vol. 55, no. 7, pp. 116–123, 2012.
- [39] I. Leyva-Mayorga et al., "Satellite edge computing for real-time and very-high resolution earth observation," *IEEE Trans. Commun.*, vol. 71, no. 10, pp. 6180–6194, Oct. 2023.
- [40] C.-X. Wang et al., "On the road to 6G: Visions, requirements, key technologies, and testbeds," *IEEE Commun. Surv. Tut.*, vol. 25, no. 2, pp. 905–974, Second Quarter 2023.
- [41] Q. Wang, X. Chen, and Q. Qi, "Energy-efficient design of satellite-terrestrial computing in 6G wireless networks," *IEEE Trans. Commun.*, vol. 72, no. 3, pp. 1759–1772, Mar. 2024.
- [42] Y. Chen, Y. Yang, J. Hu, Y. Wu, and J. Huang, "A game-theoretical approach for distributed computation offloading in LEO satellite-terrestrial edge computing systems," *IEEE Trans. Mobile Comput.*, vol. 24, no. 5, pp. 4389–4402, May 2025.